

Annotated Code Edition

Complete reference listings

Spring Boot + MySQL

React + square red-and-white UI

Windows NFC Reader Agent

Sui Move + Testnet Worker

Deployment, testing and recovery

Read the code. Understand the invariant.

File paths, responsibilities, critical annotations and tests.

Not certified for live hospital use; integration gates are explicit.

Contents

Click a chapter title to jump to its actual PDF page.

00 Contract, scope and reading conventions	4
01 Files and assembly order	5
02 Backend build and runtime configuration	6
03 Database schema and final constraints	8
04 Application bootstrap and helper rules	10
05 Security boundaries and session handling	11
06 Store: atomic commands and audit snapshots	14
07 Store: scan job lifecycle	17
08 Store: enroll, issue and return	19
09 Store: administration, queries and proof data	21
10 HTTP endpoints and safe errors	23
11 Frontend build files and request client	26
12 Shared controls and square design tokens	28

Contents

Click a chapter title to jump to its actual PDF page.

13 Frontend: registration and scanner	30
14 Frontend: staff and administrator workspace	33
15 Windows reader agent	36
16 Move package and immutable commitments	39
17 Move tests and publication procedure	41
18 Worker configuration and durable submission	43
19 Independent on-chain proof verifier	46
20 Executable unit tests and integration gates	48
21 Local setup with exact environment responsibilities	50
22 Deploy, back up and recover	52
23 Annotation index and release boundary	53
24 Approval updates, return reminders and overdue alerts	55

00 Contract, scope and reading conventions

This is the English annotated-code companion, delivered as a PDF only. File listings are complete within the reference implementation: imports, configuration, SQL, UI, reader, Move, worker and tests are included rather than replacing application methods with TODOs. Copy each listing to its named file; line numbers printed in the margin are not part of the source. Multiple listings under one file heading are consecutive parts of that file unless explicitly stated otherwise.

The reference implements registration for four categories, a deterministic mock hospital check, account login and role enforcement, review/rejection, card inventory, scan jobs, atomic issue/return, overdue calculation, account administration, a configurable loan duration, summary reporting, immutable audit snapshots, durable blockchain delivery and independent proof checking. The interface is a compact working-reference design, not a claim to reproduce the linked Figma file or a complete hospital-approved form.

Explicit implementation decisions

This edition chooses Spring Boot 3.5.16 and Java 21 to keep the familiar Jackson 2 and starter APIs internally consistent. It is a stable-version reference, not a claim that 3.5 is the newest Boot line. Do not replace only the parent with Boot 4 without adapting dependencies/imports. It uses Spring JDBC rather than the guideline's suggested JPA; SQL and transaction boundaries are deliberately visible for learning. Business mutations share a database gate row: safe and easy to reason about for a small FYP, but a throughput bottleneck to replace with ordered per-resource locks after measuring.

Because no reader/card model is confirmed, this edition explicitly uses UID-to-category inventory mapping. Admin selects category during enrollment; subsequent assignment checks use that stored category. It does not pretend to read category from UID or implement universal NDEF commands. The reader's FF CA command requires compatible PC/SC hardware. A `MOCK_UID` option is clearly labelled and must not be enabled with real data.

The hospital adapter is mock-only and accepts named synthetic fixtures. No real hospital URL is invented. The code must not be deployed for actual patient use without replacing that adapter, agreeing form/retention rules, adding perimeter rate limits and completing the release gates. The system manages passes, not physical door actuation.

Verification status

The document is authored and layout-checked; executable code is a reference for you to assemble and test. No claim is made that Spring/MySQL integration, an actual USB reader, Move compilation, Testnet publication or cloud deployment has passed in your environment. Platform secrets and published object IDs are intentionally external. Dependency installation generates lockfiles; save them after successful installation. No accounts, paid resources or blockchain deployments have been created.

Annotations use [Sxx] security, [Dxx] database, [Bxx] business, [Uxx] UI, [Rxx] reader, [Mxx] Move and [Wxx] worker. Each chapter explains meaning, purpose, importance and a concrete test. 'Critical' means removing the invariant can corrupt state or bypass authorization; 'High' means a failure can mislead users or break recovery.

01 Files and assembly order

Create a new learning directory, not on top of an existing working application. The listings are the application source; generated dependencies, Maven binaries and lockfiles are not printed. Use installed Maven 3.9+ or generate a Maven wrapper through Initializr. Prefer Java 21 and Node 24.

TEXT / LINE NUMBERS ARE NOT SOURCE

```

1 hsaas-reference/
2   backend/
3     pom.xml
4     Dockerfile
5     src/main/resources/application.yml
6     src/main/resources/db/migration/V1__schema.sql
7     src/main/java/edu/upm/hsaas/App.java
8     src/main/java/edu/upm/hsaas/Security.java
9     src/main/java/edu/upm/hsaas/Store.java
10    src/main/java/edu/upm/hsaas/Api.java
11    src/test/java/edu/upm/hsaas/RulesTest.java
12 frontend/
13   package.json, index.html, tsconfig.json, vite.config.ts
14   src/main.tsx, src/App.tsx, src/api.ts, src/index.css
15   src/components/ui/button.tsx
16 reader-agent/
17   pom.xml
18   src/main/java/ReaderAgent.java
19 sui-worker/
20   package.json, worker.mjs, verify.mjs
21 move/hsaas_audit/
22   Move.toml
23   sources/audit.move
24   tests/audit_tests.move
25 infra/compose.yaml
26 .gitignore

```

Start MySQL, then Backend, then Frontend. Test registration and authentication before starting the mock reader. Replace mock reading with actual USB only after that succeeds. Publish Move and configure the worker last. The browser never receives a device token, database password or Sui private key.

.gitignore

GITIGNORE / LINE NUMBERS ARE NOT SOURCE

```

1 **/node_modules/
2 **/target/
3 **/dist/
4 **/build/
5 **/.env
6 **/.env.*
7 !**/.env.example
8 **/secrets/
9 **/logs/
10 **/*.sql.backup

```

02 Backend build and runtime configuration

backend/pom.xml

XML / LINE NUMBERS ARE NOT SOURCE

```

1 <project xmlns="http://maven.apache.org/POM/4.0.0"
2   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
4     https://maven.apache.org/xsd/maven-4.0.0.xsd">
5   <modelVersion>4.0.0</modelVersion>
6   <parent>
7     <groupId>org.springframework.boot</groupId>
8     <artifactId>spring-boot-starter-parent</artifactId>
9     <version>3.5.16</version><relativePath/>
10  </parent>
11  <groupId>edu.upm</groupId><artifactId>hsaas</artifactId>
12  <version>1.0.0</version>
13  <properties><java.version>21</java.version></properties>
14  <dependencies>
15    <dependency><groupId>org.springframework.boot</groupId>
16      <artifactId>spring-boot-starter-web</artifactId></dependency>
17    <dependency><groupId>org.springframework.boot</groupId>
18      <artifactId>spring-boot-starter-jdbc</artifactId></dependency>
19    <dependency><groupId>org.springframework.boot</groupId>
20      <artifactId>spring-boot-starter-security</artifactId></dependency>
21    <dependency><groupId>org.springframework.boot</groupId>
22      <artifactId>spring-boot-starter-actuator</artifactId></dependency>
23    <dependency><groupId>org.flywaydb</groupId>
24      <artifactId>flyway-core</artifactId></dependency>
25    <dependency><groupId>org.flywaydb</groupId>
26      <artifactId>flyway-mysql</artifactId></dependency>
27    <dependency><groupId>com.mysql</groupId>
28      <artifactId>mysql-connector-j</artifactId><scope>runtime</scope></dependency>
29    <dependency><groupId>org.springframework.boot</groupId>
30      <artifactId>spring-boot-starter-test</artifactId><scope>test</scope></dependency>

```

XML / LINE NUMBERS ARE NOT SOURCE

```

31 </dependencies>
32 <build><finalName>app</finalName><plugins><plugin>
33   <groupId>org.springframework.boot</groupId>
34   <artifactId>spring-boot-maven-plugin</artifactId>
35 </plugin></plugins></build>
36 </project>

```

backend/src/main/resources/application.yml

YAML / LINE NUMBERS ARE NOT SOURCE

```
1 server:
2   port: ${PORT:8080}
3   servlet:
4     session:
5       timeout: 30m
6     cookie:
7       http-only: true
8       secure: ${COOKIE_SECURE:false}
9       same-site: lax
10      path: /
11  spring:
12    datasource:
13      url: ${DB_URL}
14      username: ${DB_USER}
15      password: ${DB_PASSWORD}
16      hikari:
17        maximum-pool-size: 5
18        connection-init-sql: "SET time_zone = '+00:00'"
19    flyway:
20      enabled: true
21    jackson:
22      deserialization:
23        fail-on-unknown-properties: true
24    management:
25      endpoints:
26        web:
27          exposure:
28            include: health
29    endpoint:
30      health:
```

YAML / LINE NUMBERS ARE NOT SOURCE

```
31      show-details: never
```

Meaning: configuration enters through process environment, not source constants. Purpose: the same code can use isolated local/test databases. Importance: High; COOKIE_SECURE must be true behind public HTTPS. In-memory sessions deliberately expire on restart; add Spring Session JDBC before running multiple Backend instances.

03 Database schema and final constraints

backend/src/main/resources/db/migration/V1__schema.sql

SQL / LINE NUMBERS ARE NOT SOURCE

```

1 CREATE TABLE gate (id INT PRIMARY KEY);
2 INSERT INTO gate VALUES (1);
3
4 CREATE TABLE users (
5   username VARCHAR(50) PRIMARY KEY,
6   password_hash VARCHAR(100) NOT NULL,
7   role VARCHAR(20) NOT NULL,
8   active BOOLEAN NOT NULL DEFAULT TRUE,
9   CHECK (role IN ('ADMIN','COUNTER_STAFF'))
10 );
11 CREATE TABLE settings (
12   id INT PRIMARY KEY, hours INT NOT NULL,
13   CHECK (hours BETWEEN 1 AND 168)
14 );
15 INSERT INTO settings VALUES (1,8);
16
17 CREATE TABLE registrations (
18   id CHAR(36) PRIMARY KEY,
19   category VARCHAR(20) NOT NULL,
20   data TEXT NOT NULL,
21   state VARCHAR(12) NOT NULL DEFAULT 'SUBMITTED',
22   reviewer VARCHAR(50), reason VARCHAR(300),
23   created_at TIMESTAMP(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
24   CHECK (category IN ('EXECUTIVE','PENJAGA','VENDOR','CONTRACTOR')),
25   CHECK (state IN ('SUBMITTED','VERIFIED','REJECTED')),
26   INDEX ix_queue (state,created_at)
27 );
28 CREATE TABLE cards (
29   uid VARCHAR(32) PRIMARY KEY,
30   category VARCHAR(20) NOT NULL,

```

SQL / LINE NUMBERS ARE NOT SOURCE

```

31   state VARCHAR(12) NOT NULL DEFAULT 'AVAILABLE',
32   CHECK (category IN ('EXECUTIVE','PENJAGA','VENDOR','CONTRACTOR')),
33   CHECK (state IN ('AVAILABLE','ISSUED','DISABLED'))
34 );
35 CREATE TABLE assignments (
36   id CHAR(36) PRIMARY KEY,
37   registration_id CHAR(36) NOT NULL,
38   uid VARCHAR(32) NOT NULL,
39   issued_by VARCHAR(50) NOT NULL,
40   issued_at TIMESTAMP(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
41   due_at TIMESTAMP(6) NOT NULL,
42   returned_at TIMESTAMP(6) NULL,
43   returned_by VARCHAR(50),
44   active_uid VARCHAR(32) GENERATED ALWAYS AS
45     (CASE WHEN returned_at IS NULL THEN uid ELSE NULL END) STORED,
46   active_registration CHAR(36) GENERATED ALWAYS AS
47     (CASE WHEN returned_at IS NULL THEN registration_id ELSE NULL END) STORED,
48   UNIQUE KEY uq_active_card (active_uid),
49   UNIQUE KEY uq_active_registration (active_registration),
50   FOREIGN KEY (uid) REFERENCES cards(uid),
51   FOREIGN KEY (registration_id) REFERENCES registrations(id),
52   INDEX ix_due (returned_at,due_at)

```


SQL / LINE NUMBERS ARE NOT SOURCE

```

53 );
54 CREATE TABLE scans (
55   id CHAR(36) PRIMARY KEY,
56   actor VARCHAR(50) NOT NULL,
57   purpose VARCHAR(12) NOT NULL,
58   target CHAR(36) NOT NULL DEFAULT '',
59   state VARCHAR(12) NOT NULL DEFAULT 'WAITING',
60   uid VARCHAR(32), lease_token CHAR(36),

```

SQL / LINE NUMBERS ARE NOT SOURCE

```

61   expires_at TIMESTAMP(6) NOT NULL,
62   CHECK (purpose IN ('ENROLL','ISSUE')),
63   CHECK (state IN ('WAITING','READING','READY','ERROR','CANCELLED','CONSUMED'))
64 );
65 CREATE TABLE commands (
66   actor VARCHAR(100) NOT NULL,
67   operation VARCHAR(50) NOT NULL,
68   command_key CHAR(36) NOT NULL,
69   request_hash CHAR(64) NOT NULL,
70   response TEXT NOT NULL,
71   PRIMARY KEY (actor,operation,command_key)
72 );
73 CREATE TABLE audit_events (
74   id CHAR(36) PRIMARY KEY,
75   event_type VARCHAR(40) NOT NULL,
76   payload TEXT NOT NULL,
77   hash CHAR(64) NOT NULL,
78   created_at TIMESTAMP(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6)
79 );
80 CREATE TABLE outbox (
81   event_id CHAR(36) PRIMARY KEY,
82   state VARCHAR(16) NOT NULL DEFAULT 'PENDING',
83   tx_bytes MEDIUMTEXT, signature TEXT,
84   digest VARCHAR(100), attempts INT NOT NULL DEFAULT 0,
85   error_code VARCHAR(60),
86   FOREIGN KEY (event_id) REFERENCES audit_events(id)
87 );

```

[D01] Critical: the two generated unique columns enforce one active card and one active registration at the database layer. They remain effective even if two callers pass an earlier application check. Test by directly attempting two open rows with the same UID.

[D02] Critical: commands, business rows, audit_events and outbox are committed together. A duplicate command returns its original response. The gate row serializes these short transactions, trading throughput for straightforward correctness. Do not hold it across NFC or Sui network waits.

The code derives OVERDUE from an open assignment's due_at rather than maintaining a second scheduler-owned card state. This avoids return/scheduler races and shows overdue immediately; there is no separate overdue-event anchor in this reference. Historical issue/return events remain available. Add an explicit overdue event only if the approved requirement needs it.

04 Application bootstrap and helper rules

backend/src/main/java/edu/upm/hsaas/App.java

JAVA / LINE NUMBERS ARE NOT SOURCE

```

1 package edu.upm.hsaas;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5 import org.springframework.boot.CommandLineRunner;
6 import org.springframework.context.annotation.Bean;
7 import org.springframework.jdbc.core.JdbcTemplate;
8 import org.springframework.security.crypto.password.PasswordEncoder;
9
10 @SpringBootApplication
11 public class App {
12     public static void main(String[] args) {
13         SpringApplication.run(App.class, args);
14     }
15     @Bean CommandLineRunner bootstrap(JdbcTemplate db, PasswordEncoder encoder) {
16         return args -> {
17             String secret = System.getenv("BOOTSTRAP_PASSWORD");
18             if (secret == null) return;
19             if (secret.length() < 16) throw new IllegalStateException("Weak bootstrap");
20             // [S01] No shared default password and no overwrite of existing accounts.
21             db.update("""
22                 INSERT INTO users(username,password_hash,role)
23                 SELECT 'admin',?, 'ADMIN' WHERE NOT EXISTS (SELECT 1 FROM users)
24                 """, encoder.encode(secret));
25         });
26     }
27 }

```

Run one instance for initial bootstrap; remove BOOTSTRAP_PASSWORD after creation. The initialization query is not a substitute for a multi-instance provisioning workflow. Login uses username admin and your own injected password.

[S01] Critical: no hard-coded default administrator credential is distributed. Losing the configured password requires an authorized reset, not an undocumented backdoor.

05 Security boundaries and session handling

backend/src/main/java/edu/upm/hsaas/Security.java

JAVA / LINE NUMBERS ARE NOT SOURCE

```

1 package edu.upm.hsaas;
2
3 import jakarta.servlet.FilterChain;
4 import jakarta.servlet.ServletException;
5 import jakarta.servlet.http.HttpServletRequest;
6 import jakarta.servlet.http.HttpServletResponse;
7 import java.io.IOException;
8 import java.nio.charset.StandardCharsets;
9 import java.security.MessageDigest;
10 import org.springframework.context.annotation.Bean;
11 import org.springframework.context.annotation.Configuration;
12 import org.springframework.core.annotation.Order;
13 import org.springframework.jdbc.core.JdbcTemplate;
14 import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
15 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
16 import org.springframework.security.config.http.SessionCreationPolicy;
17 import org.springframework.security.core.context.SecurityContextHolder;
18 import org.springframework.security.core.userdetails.*;
19 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
20 import org.springframework.security.crypto.password.PasswordEncoder;
21 import org.springframework.security.web.SecurityFilterChain;
22 import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
23 import org.springframework.web.filter.OncePerRequestFilter;
24
25 @Configuration
26 public class Security {
27     @Bean PasswordEncoder encoder() { return new BCryptPasswordEncoder(12); }
28     @Bean UserDetailsServiceImpl accounts(JdbcTemplate db) {
29         return name -> {
30             var rows = db.queryForList("SELECT * FROM users WHERE username=?", name);

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

31         if (rows.isEmpty()) throw new UsernameNotFoundException("Unknown account");
32         var r = rows.getFirst();
33         return User.withUsername(name).password((String) r.get("password_hash"))
34             .roles((String) r.get("role"))
35             .disabled(!Boolean.TRUE.equals(r.get("active"))).build();
36     };
37 }
38 static void error(HttpServletResponse res, int status, String code)
39 throws IOException {
40     res.setStatus(status); res.setContentType("application/json");
41     res.getWriter().write("{\"code\":\"" + code + "\"}");
42 }
43 @Bean @Order(1)
44 SecurityFilterChain device(HttpSecurity http) throws Exception {
45     String token = System.getenv("DEVICE_TOKEN");
46     if (token == null || token.length() < 32)
47         throw new IllegalStateException("DEVICE_TOKEN needs 32+ random characters");
48     // [S02] CSRF exemption is limited to bearer-only device routes.
49     http.securityMatcher("/api/device/**")
50         .csrf(c -> c.disable())
51         .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
52         .authorizeHttpRequests(a -> a.anyRequest().hasRole("DEVICE"))

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

53 .exceptionHandling(e -> e.authenticationEntryPoint(
54 (req,res,ex) -> error(res,401,"DEVICE_AUTH")))
55 .addFilterBefore(new OncePerRequestFilter() {
56     protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
57         FilterChain chain) throws IOException, ServletException {
58         String value = req.getHeader("Authorization");
59         byte[] actual = (value == null ? "" : value).getBytes(StandardCharsets.UTF_8);
60         byte[] expected = ("Bearer " + token).getBytes(StandardCharsets.UTF_8);

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

61     if (MessageDigest.isEqual(actual,expected)) {
62         var user = User.withUsername("reader-1").password("").roles("DEVICE").build();
63         var auth = new UsernamePasswordAuthenticationToken(user,null,user.getAuthorities());
64         SecurityContextHolder.getContext().setAuthentication(auth);
65     }
66     chain.doFilter(req,res);
67 }
68 }, UsernamePasswordAuthenticationFilter.class);
69 return http.build();
70 }
71 @Bean @Order(2)
72 SecurityFilterChain browser(HttpSecurity http, UserDetailsService accounts)
73 throws Exception {
74     http.authorizeHttpRequests(a -> a
75         .requestMatchers("/api/public/**","/api/auth/csrf","/actuator/health").permitAll()
76         .requestMatchers("/api/admin/**").hasRole("ADMIN")
77         .requestMatchers("/api/staff/**").hasAnyRole("ADMIN","COUNTER_STAFF")
78         .anyRequest().authenticated()
79         .formLogin(f -> f.loginProcessingUrl("/api/auth/login").permitAll()
80             .successHandler((req,res,auth) -> {
81                 res.setContentType("application/json"); res.getWriter().write("{\"ok\":true}");
82             }).failureHandler((req,res,ex) -> error(res,401,"LOGIN_FAILED")))
83         .logout(l -> l.logoutUrl("/api/auth/logout").deleteCookies("JSESSIONID")
84             .logoutSuccessHandler((req,res,auth) -> res.setStatus(204)))
85         .exceptionHandling(e -> e
86             .authenticationEntryPoint((req,res,ex) -> error(res,401,"LOGIN_REQUIRED"))
87             .accessDeniedHandler((req,res,ex) -> error(res,403,"ACCESS_DENIED")))
88         .addFilterBefore(new OncePerRequestFilter() {
89             protected void doFilterInternal(HttpServletRequest req,HttpServletResponse res,
90                 FilterChain chain) throws IOException,ServletException {

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

91     // [S03] Reload current role/active state for existing sessions.
92     var auth = SecurityContextHolder.getContext().getAuthentication();
93     if (auth != null && auth.isAuthenticated()) {
94         try {
95             var fresh = accounts.loadUserByUsername(auth.getName());
96             if (!fresh.isEnabled()) throw new UsernameNotFoundException("Disabled");
97             SecurityContextHolder.getContext().setAuthentication(
98                 new UsernamePasswordAuthenticationToken(fresh,null,fresh.getAuthorities()));
99         } catch (UsernameNotFoundException ex) {
100             SecurityContextHolder.clearContext();
101             var session = req.getSession(false);
102             if (session != null) session.invalidate();
103             error(res,401,"SESSION_REVOKED"); return;
104         }
105     }
106     res.setHeader("Cache-Control","no-store");
107     chain.doFilter(req,res);

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```
108     }
109     }, UsernamePasswordAuthenticationFilter.class);
110     return http.build();
111 }
112 }
```

[S02] Critical: a device token can only access the device API, not enroll cards or issue passes as a human. The reference has one configured counter reader. Multi-counter deployment requires per-device records, revocation and job/device binding; do not duplicate this token across a hospital.

[S03] High: a disabled or demoted account must not retain old privileges through a cached session. Password reset is not included in account editing; handle resets by a separate reviewed workflow that invalidates sessions. CSRF remains enabled for browser writes, including anonymous registration and login.

Tests: wrong device token yields 401; missing CSRF yields 403; staff cannot call admin endpoints; disabling a user ends their ability to make requests. Rate-limit login and anonymous endpoints at the reverse proxy before any public deployment.

06 Store: atomic commands and audit snapshots

backend/src/main/java/edu/upm/hsaas/Store.java

This file continues through chapters 07-09. Concatenate the four Java listings in order; there is one package declaration, one class and one closing class brace at the end of chapter 09.

JAVA / LINE NUMBERS ARE NOT SOURCE

```

1 package edu.upm.hsaas;
2
3 import com.fasterxml.jackson.core.type.TypeReference;
4 import com.fasterxml.jackson.databind.ObjectMapper;
5 import java.nio.charset.StandardCharsets;
6 import java.security.MessageDigest;
7 import java.sql.Timestamp;
8 import java.time.Instant;
9 import java.util.*;
10 import java.util.function.Supplier;
11 import org.springframework.http.HttpStatus;
12 import org.springframework.jdbc.core.JdbcTemplate;
13 import org.springframework.security.crypto.password.PasswordEncoder;
14 import org.springframework.stereotype.Service;
15 import org.springframework.transaction.support.TransactionTemplate;
16 import org.springframework.web.server.ResponseStatusException;
17
18 @Service
19 public class Store {
20     final JdbcTemplate db;
21     final TransactionTemplate tx;
22     final ObjectMapper json;
23     final PasswordEncoder passwords;
24     public Store(JdbcTemplate db,TransactionTemplate tx,ObjectMapper json,
25         PasswordEncoder passwords) {
26         this.db=db; this.tx=tx; this.json=json; this.passwords=passwords;
27     }
28     static final Set<String> CATEGORIES = Set.of(
29         "EXECUTIVE","PENJAGA","VENDOR","CONTRACTOR");
30     static String id() { return UUID.randomUUID().toString(); }

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

31 static void require(boolean ok,String code) {
32     if (!ok) throw new ResponseStatusException(HttpStatus.CONFLICT,code);
33 }
34 static String value(Map<String,String> m,String key,int max) {
35     String s=m.getOrDefault(key,"").trim();
36     require(!s.isEmpty() && s.length()<=max,"INVALID_"+key.toUpperCase());
37     return s;
38 }
39 static String uid(String raw) {
40     String u=raw.replace(" ","").replace(":","").toUpperCase(Locale.ROOT);
41     require(u.matches("(?:[0-9A-F]{2}){4,10}"),"INVALID_UID"); return u;
42 }
43 static String sha(String s) {
44     try {
45         return HexFormat.of().formatHex(MessageDigest.getInstance("SHA-256")
46             .digest(s.getBytes(StandardCharsets.UTF_8)));
47     } catch (Exception e) { throw new IllegalStateException(e); }
48 }
49 String encode(Object x) {

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

50     try { return json.writeValueAsString(x); }
51     catch (Exception e) { throw new IllegalStateException(e); }
52 }
53 Map<String,Object> decode(String s) {
54     try { return json.readValue(s,new TypeReference<Map<String,Object>>() {}); }
55     catch (Exception e) { throw new IllegalStateException(e); }
56 }
57 Map<String,Object> one(String sql,Object... args) {
58     var rows=db.queryForList(sql,args);
59     require(rows.size()==1,"NOT_FOUND"); return rows.getFirst();
60 }

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

61 public Map<String,Object> command(String actor,String op,String key,
62     Map<String,String> body,Supplier<Map<String,Object>> action) {
63     require(key != null && key.matches("[0-9a-fA-F-]{36}"),"INVALID_COMMAND_KEY");
64     require(body.size()<=20,"TOO_MANY_FIELDS");
65     body.values().forEach(v -> require(v!=null && v.length()<=1000,"INPUT_TOO_LONG"));
66     String hash=sha(encode(new TreeMap<>(body)));
67     return tx.execute(status -> {
68         // [D03] One short DB transaction; no external network calls inside it.
69         db.queryForObject("SELECT id FROM gate WHERE id=1 FOR UPDATE",Integer.class);
70         var old=db.queryForList("""
71             SELECT request_hash,response FROM commands
72             WHERE actor=? AND operation=? AND command_key=?
73             """,actor,op,key);
74         if (!old.isEmpty()) {
75             require(hash.equals(old.getFirst().get("request_hash")), "KEY_REUSED");
76             return decode((String)old.getFirst().get("response"));
77         }
78         var result=action.get();
79         db.update("INSERT INTO commands VALUES (?, ?, ?, ?, ?)",
80             actor,op,key,hash,encode(result));
81         return result;
82     });
83 }
84 void audit(String type,String resource,String actor) {
85     String eventId=id();
86     // [D04] Snapshot format v1: explicit keys, UTC text, random nonce.
87     var payload=new TreeMap<String,Object>();
88     payload.put("version",1); payload.put("eventId",eventId);
89     payload.put("type",type); payload.put("resource",resource);
90     payload.put("actor",actor); payload.put("time",Instant.now().toString());

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

91     payload.put("nonce",id());
92     String bytes=encode(payload);
93     db.update("INSERT INTO audit_events(id,event_type,payload,hash) VALUES (?, ?, ?, ?)",
94         eventId,type,bytes,sha(bytes));
95     db.update("INSERT INTO outbox(event_id) VALUES (?)",eventId);
96 }
97 public String hospital(Map<String,String> b) {
98     String mrn=value(b,"mrn",40),ward=value(b,"ward",40);
99     if (mrn.equals("DEMO-TIMEOUT")) return "UNAVAILABLE";
100     return mrn.equals("DEMO-0001") && ward.equals("WARD-A") ? "VALID" : "INVALID";
101 }
102 public Map<String,Object> register(Map<String,String> b) {
103     String category=value(b,"category",20);
104     require(CATEGORIES.contains(category),"CATEGORY_UNKNOWN");

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

105     var clean=new TreeMap<String,String>();
106     clean.put("name",value(b,"name",120)); clean.put("phone",value(b,"phone",32));
107     clean.put("destination",value(b,"destination",80));
108     clean.put("purpose",value(b,"purpose",200));
109     require("yes".equals(b.get("consent")), "NOTICE_REQUIRED");
110     if (category.equals("PENJAGA")) {
111         // [B01] Validate again on final submit, never trust a browser checkmark.
112         String result=hospital(b); require(result.equals("VALID"), "MRN_"+result);
113         clean.put("mrn",value(b,"mrn",40)); clean.put("ward",value(b,"ward",40));
114         clean.put("mrnSource", "MOCK");
115     } else if (category.equals("VENDOR") || category.equals("CONTRACTOR")) {
116         clean.put("organization",value(b,"organization",120));
117     }
118     String reference=id();
119     db.update("INSERT INTO registrations(id,category,data) VALUES (?, ?, ?)",
120         reference,category,encode(clean));

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

121     audit("REGISTERED",reference,"PUBLIC");
122     return Map.of("reference",reference,"state","SUBMITTED","hospitalMode","MOCK");
123 }
124 public Map<String,Object> review(String reg,Map<String,String> b,String actor) {
125     var row=one("SELECT * FROM registrations WHERE id=?",reg);
126     require("SUBMITTED".equals(row.get("state")), "ALREADY_REVIEWED");
127     String decision=value(b,"decision",12);
128     require(Set.of("VERIFIED","REJECTED").contains(decision), "BAD_DECISION");
129     String reason=decision.equals("REJECTED") ? value(b,"reason",300) : "";
130     db.update("UPDATE registrations SET state=?,reviewer=?,reason=? WHERE id=?",
131         decision,actor,reason,reg); audit(decision,reg,actor);
132     return Map.of("id",reg,"state",decision);
133 }

```

[D03] Critical: command hashing and response persistence make retried mutations deterministic. The gate makes the existence check safe across instances using the same database. Anonymous commands are scoped by browser session rather than a caller-supplied actor.

[D04] High: this v1 snapshot proves the specified event metadata, not every visitor field. No raw UID or MRN is published to Sui; even eventId is random. Exact serialized bytes are retained so verification does not depend on later serializer changes. Expand the versioned snapshot deliberately if form-field integrity must also be proven.

[B01] Critical: mock validation is repeated during submission. When substituting a real network API, perform remote validation before entering command(), persist bound short-lived evidence and recheck that evidence inside the transaction. Do not add hospital network waits under the global database lock.

07 Store: scan job lifecycle

Store.java, continuation

JAVA / LINE NUMBERS ARE NOT SOURCE

```

1  public Map<String,Object> startScan(Map<String,String> b,String actor,boolean admin) {
2      String purpose=value(b,"purpose",12);
3      require(Set.of("ENROLL","ISSUE").contains(purpose),"BAD_SCAN_PURPOSE");
4      require(!purpose.equals("ENROLL") || admin,"ADMIN_REQUIRED");
5      String target=purpose.equals("ISSUE") ? value(b,"target",36) : "";
6      if (purpose.equals("ISSUE")) {
7          var r=one("SELECT state FROM registrations WHERE id=?",target);
8          require("VERIFIED".equals(r.get("state")), "NOT_VERIFIED");
9      }
10     db.update("""
11         UPDATE scans SET state='ERROR'
12         WHERE state IN ('WAITING','READING','READY') AND expires_at<CURRENT_TIMESTAMP
13         """);
14     Integer active=db.queryForObject("""
15         SELECT COUNT(*) FROM scans WHERE state IN ('WAITING','READING','READY')
16         """,Integer.class);
17     require(active==0,"READER_BUSY");
18     String job=id();
19     db.update("""
20         INSERT INTO scans(id,actor,purpose,target,expires_at)
21         VALUES (?,?,,?,DATE_ADD(CURRENT_TIMESTAMP,INTERVAL 90 SECOND))
22         """,job,actor,purpose,target);
23     return Map.of("id",job,"state","WAITING");
24 }
25 public Map<String,Object> scan(String id,String actor) {
26     var row=one("SELECT * FROM scans WHERE id=? AND actor=?",id,actor);
27     if (((Timestamp)row.get("expires_at")).toInstant().isBefore(Instant.now()))
28         && !Set.of("CONSUMED","CANCELLED").contains(row.get("state"))) {
29         row.put("state","ERROR");
30     }

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

31     row.remove("lease_token"); return row;
32 }
33 public Map<String,Object> cancel(String id,String actor) {
34     db.update("""
35         UPDATE scans SET state='CANCELLED'
36         WHERE id=? AND actor=? AND state IN ('WAITING','READING','READY')
37         """,id,actor);
38     return Map.of("ok",true);
39 }
40 public Map<String,Object> claim() {
41     var rows=db.queryForList("""
42         SELECT id FROM scans WHERE state='WAITING'
43         AND expires_at>CURRENT_TIMESTAMP ORDER BY expires_at LIMIT 1
44         """);
45     if (rows.isEmpty()) return Map.of("state","IDLE");
46     String job=(String)rows.getFirst().get("id"),lease=id();
47     db.update("UPDATE scans SET state='READING',lease_token=? WHERE id=?",lease,job);
48     return Map.of("state","READING","id",job,"lease",lease);
49 }
50 public Map<String,Object> result(String job,Map<String,String> b) {
51     var row=one("SELECT * FROM scans WHERE id=?",job);
52     require("READING".equals(row.get("state")), "SCAN_NOT_READING");

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

53     require(value(b, "lease", 36).equals(row.get("lease_token")), "SCAN_LEASE");
54     require(((Timestamp)row.get("expires_at")).toInstant().isAfter(Instant.now()),
55         "SCAN_EXPIRED");
56     boolean ok="READY".equals(b.get("state"));
57     String u=ok ? uid(value(b, "uid", 32)) : null;
58     db.update("UPDATE scans SET state=?,uid=? WHERE id=?", ok?"READY":"ERROR", u, job);
59     return Map.of("ok", true);
60 }

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

61 Map<String, Object> usable(Map<String, String> b, String actor, String purpose, String target) {
62     var row=one("SELECT * FROM scans WHERE id=?", value(b, "scanId", 36));
63     // [R01] Evidence belongs to this actor, purpose, target and unexpired job.
64     require(actor.equals(row.get("actor")) && purpose.equals(row.get("purpose"))
65         && target.equals(row.get("target")), "SCAN_CONTEXT");
66     require("READY".equals(row.get("state")), "SCAN_NOT_READY");
67     require(((Timestamp)row.get("expires_at")).toInstant().isAfter(Instant.now()),
68         "SCAN_EXPIRED");
69     return row;
70 }
71 void consume(Map<String, Object> scan) {
72     db.update("UPDATE scans SET state='CONSUMED' WHERE id=?", scan.get("id"));
73 }

```

[R01] Critical: a valid-looking UID alone is not enough. Backend accepts the result only in the initiating operation's context, then consumes it atomically. Failed/cancelled/expired jobs cannot be reused. Test submitting an old scan, another staff member's scan and an enrollment scan to the issue endpoint.

The one-reader implementation lets an expired READING job fail rather than assigning it to another agent. Staff explicitly retries with a new job. That is an intentional safe recovery choice; do not silently reuse a lease after a crash. Job polling does not extend the 90-second expiry.

08 Store: enroll, issue and return

Store.java, continuation

JAVA / LINE NUMBERS ARE NOT SOURCE

```

1  public Map<String,Object> enroll(Map<String,String> b,String actor) {
2      var scan=usable(b,actor,"ENROLL","");
3      String category=value(b,"category",20),u=(String)scan.get("uid");
4      require(CATEGORIES.contains(category),"CATEGORY_UNKNOWN");
5      require(db.queryForList("SELECT uid FROM cards WHERE uid=?",u).isEmpty(),
6          "DUPLICATE_CARD");
7      db.update("INSERT INTO cards(uid,category) VALUES (?,?)",u,category);
8      consume(scan); audit("CARD_ENROLLED",sha(u),actor);
9      return Map.of("uid",u,"state","AVAILABLE");
10 }
11 public Map<String,Object> issue(Map<String,String> b,String actor) {
12     String reg=value(b,"registrationId",36);
13     var r=one("SELECT * FROM registrations WHERE id=?",reg);
14     require("VERIFIED".equals(r.get("state")),"NOT_VERIFIED");
15     var scan=usable(b,actor,"ISSUE",reg);
16     String u=(String)scan.get("uid");
17     var cards=db.queryForList("SELECT * FROM cards WHERE uid=?",u);
18     require(!cards.isEmpty(),"CARD_UNREGISTERED");
19     var card=cards.getFirst();
20     // [B02] The database, not the client, supplies authoritative card state.
21     require("AVAILABLE".equals(card.get("state")),"CARD_UNAVAILABLE");
22     require(r.get("category").equals(card.get("category")),"CATEGORY_MISMATCH");
23     require(db.queryForList("
24         SELECT id FROM assignments WHERE registration_id=? AND returned_at IS NULL
25         """,reg).isEmpty(),"ALREADY_ASSIGNED");
26     String assignment=id();
27     Integer hours=db.queryForObject("SELECT hours FROM settings WHERE id=1",Integer.class);
28     db.update("
29         INSERT INTO assignments(id,registration_id,uid,issued_by,due_at)
30         VALUES (?,?,,?,DATE_ADD(CURRENT_TIMESTAMP,INTERVAL ? HOUR))

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

31     """,assignment,reg,u,actor,hours);
32     db.update("UPDATE cards SET state='ISSUED' WHERE uid=?",u);
33     consume(scan); audit("ISSUED",assignment,actor);
34     return Map.of("id",assignment,"uid",u,"proof","PENDING");
35 }
36 public Map<String,Object> returned(String assignment,String actor) {
37     var a=one("SELECT * FROM assignments WHERE id=?",assignment);
38     if (a.get("returned_at")!=null) return Map.of("id",assignment,"state","RETURNED");
39     db.update("
40         UPDATE assignments SET returned_at=CURRENT_TIMESTAMP,returned_by=? WHERE id=?
41         """,actor,assignment);
42     db.update("UPDATE cards SET state='AVAILABLE' WHERE uid=?",a.get("uid"));
43     audit("RETURNED",assignment,actor);
44     return Map.of("id",assignment,"state","RETURNED");
45 }
46 public Map<String,Object> cardState(String raw,Map<String,String> b,String actor) {
47     String u=uid(raw),next=value(b,"state",12);
48     var c=one("SELECT state FROM cards WHERE uid=?",u);
49     require(!"ISSUED".equals(c.get("state")),"CARD_IN_USE");
50     require(Set.of("AVAILABLE","DISABLED").contains(next),"BAD_CARD_STATE");
51     db.update("UPDATE cards SET state=? WHERE uid=?",next,u);
52     audit("CARD_"+next,sha(u),actor); return Map.of("ok",true);

```

```
JAVA / LINE NUMBERS ARE NOT SOURCE
```

```
53 }
```

[B02] Critical: UI compatibility checks are only feedback. Server-side state/category checks prevent assignment using forged JSON or stale screens. The unique constraints are an additional defense. Enrollment never silently overwrites an existing category.

Return is an authenticated staff action selected from an assignment record and confirmed in the UI; it does not fabricate a successful NFC scan. If policy requires mandatory scan-on-return, add RETURN scan context and enforce it before accepting that command. The reference's original assignment flow still requires a real or explicitly simulated scan.

Test same-card concurrent issue, same-registration concurrent issue, duplicate enrollment, mismatched category, repeated return and injected audit-write failure. Observe final row counts, not only UI messages.

09 Store: administration, queries and proof data

Store.java, final continuation

JAVA / LINE NUMBERS ARE NOT SOURCE

```

1 public Map<String,Object> user(Map<String,String> b,String actor) {
2     String name=value(b,"username",50),role=value(b,"role",20);
3     require(name.matches("[a-zA-Z0-9._-]{3,50}"),"BAD_USERNAME");
4     require(Set.of("ADMIN","COUNTER_STAFF").contains(role),"BAD_ROLE");
5     boolean active="true".equals(b.get("active"));
6     var old=db.queryForList("SELECT * FROM users WHERE username=?",name);
7     if (old.isEmpty()) {
8         String password=value(b,"password",72);
9         require(password.length()>=16 && password.getBytes(StandardCharsets.UTF_8).length<=72,
10             "PASSWORD_LENGTH");
11         db.update("INSERT INTO users VALUES (?,?=?,?)",name,passwords.encode(password),role,active);
12     } else {
13         var u=old.getFirst();
14         boolean wasAdmin="ADMIN".equals(u.get("role")) && Boolean.TRUE.equals(u.get("active"));
15         if (wasAdmin && (!active || !role.equals("ADMIN"))) {
16             int count=db.queryForObject("""
17                 SELECT COUNT(*) FROM users WHERE role='ADMIN' AND active=TRUE
18                 """,Integer.class);
19             require(count>1,"LAST_ADMIN");
20         }
21         require(b.getOrDefault("password","").isEmpty(),"RESET_NOT_SUPPORTED_HERE");
22         db.update("UPDATE users SET role=?,active=? WHERE username=?",role,active,name);
23     }
24     audit("USER_CHANGED",name,actor); return Map.of("ok",true);
25 }
26 public Map<String,Object> settings(Map<String,String> b,String actor) {
27     int hours;
28     try { hours=Integer.parseInt(value(b,"hours",3)); }
29     catch (NumberFormatException e) { throw new ResponseStatusException(
30         HttpStatus.BAD_REQUEST,"INVALID_HOURS"); }

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

31     require(hours>=1 && hours<=168,"INVALID_HOURS");
32     db.update("UPDATE settings SET hours=? WHERE id=1",hours);
33     audit("SETTINGS_CHANGED",Integer.toString(hours),actor); return Map.of("hours",hours);
34 }
35 public List<Map<String,Object>> registrations(int page) {
36     require(page>=0 && page<=10000,"INVALID_PAGE");
37     return db.queryForList("""
38         SELECT id,category,state,created_at FROM registrations
39         ORDER BY created_at DESC,id LIMIT 20 OFFSET ?
40         """,page*20);
41 }
42 public Map<String,Object> detail(String id) {
43     var r=one("SELECT * FROM registrations WHERE id=?",id);
44     r.put("data",decode((String)r.get("data"))); return r;
45 }
46 public List<Map<String,Object>> assignments(int page) {
47     require(page>=0 && page<=10000,"INVALID_PAGE");
48     return db.queryForList("""
49         SELECT id,registration_id,uid,issued_at,due_at,returned_at,
50         CASE WHEN returned_at IS NOT NULL THEN 'RETURNED'
51         WHEN due_at<CURRENT_TIMESTAMP THEN 'OVERDUE' ELSE 'ISSUED' END AS state
52         FROM assignments ORDER BY issued_at DESC,id LIMIT 20 OFFSET ?

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

53     """,page*20);
54 }
55 public Map<String,Object> summary() {
56     return Map.of("registrations",db.queryForList("""
57         SELECT category,COUNT(*) AS total FROM registrations GROUP BY category
58         """),"queue",db.queryForObject(
59         "SELECT COUNT(*) FROM registrations WHERE state='SUBMITTED'",Long.class),
60         "active",db.queryForObject(

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

61         "SELECT COUNT(*) FROM assignments WHERE returned_at IS NULL",Long.class),
62         "overdue",db.queryForObject("""
63         SELECT COUNT(*) FROM assignments
64         WHERE returned_at IS NULL AND due_at<CURRENT_TIMESTAMP
65         """,Long.class));
66 }
67 public List<Map<String,Object>> proofs(int page) {
68     require(page>=0 && page<=10000,"INVALID_PAGE");
69     return db.queryForList("""
70     SELECT a.id,a.event_type,a.hash,a.created_at,o.state,o.digest,o.error_code
71     FROM audit_events a JOIN outbox o ON a.id=o.event_id
72     ORDER BY a.created_at DESC,a.id LIMIT 20 OFFSET ?
73     """,page*20);
74 }
75 public Map<String,Object> proof(String event) {
76     var a=one("SELECT * FROM audit_events WHERE id=?",event);
77     a.put("localIntegrity",sha((String)a.get("payload")).equals(a.get("hash")));
78     return a;
79 }
80 }

```

Account editing protects the last active administrator and never returns hashes in list endpoints. Reports intentionally use current operational counts and category totals, not an invented historical attendance metric. Personal registration details are returned only by the staff detail route, not public references or queue rows.

The proof endpoint checks local snapshot consistency only. It must not label an event 'verified on-chain'; chapter 19's independent verifier checks the transaction event and expected registry/package.

10 HTTP endpoints and safe errors

backend/src/main/java/edu/upm/hsaas/Api.java

JAVA / LINE NUMBERS ARE NOT SOURCE

```

1 package edu.upm.hsaas;
2
3 import jakarta.servlet.http.HttpSession;
4 import java.util.*;
5 import java.util.function.Supplier;
6 import org.springframework.dao.DataIntegrityViolationException;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.security.core.Authentication;
9 import org.springframework.security.web.csrf.CsrfToken;
10 import org.springframework.web.bind.annotation.*;
11 import org.springframework.web.server.ResponseStatusException;
12
13 @RestController
14 public class Api {
15     final Store s;
16     public Api(Store s) { this.s=s; }
17     boolean admin(Authentication a) {
18         return a.getAuthorities().stream().anyMatch(x -> x.getAuthority().equals("ROLE_ADMIN"));
19     }
20     Map<String,Object> run(Authentication a,String op,String key,
21         Map<String,String> b,Supplier<Map<String,Object>> action) {
22         return s.command(a.getName(),op,key,b,action);
23     }
24     @GetMapping("/api/auth/csrf") Object csrf(CsrfToken t) {
25         return Map.of("token",t.getToken(),"headerName",t.getHeaderName());
26     }
27     @GetMapping("/api/auth/me") Object me(Authentication a) {
28         return Map.of("username",a.getName(),"admin",admin(a));
29     }
30     @GetMapping("/api/public/config") Object config() {

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

31     return Map.of("categories",Store.CATEGORIES,"hospitalMode","MOCK",
32         "categorySource","INVENTORY_MAPPING","readerMode",System.getenv()
33         .getOrDefault("READER_LABEL","PCSC / verify device configuration"));
34 }
35 @PostMapping("/api/public/mrn") Object mrn(@RequestBody Map<String,String> b) {
36     return Map.of("result",s.hospital(b),"mode","MOCK");
37 }
38 @PostMapping("/api/public/registrations") Object register(
39     @RequestBody Map<String,String> b,@RequestHeader("Idempotency-Key") String key,
40     HttpSession session) {
41     return s.command("public:"+session.getId(),"register",key,b,() -> s.register(b));
42 }
43 @GetMapping("/api/staff/registrations") Object registrations(
44     @RequestParam(defaultValue="0") int page) { return s.registrations(page); }
45 @GetMapping("/api/staff/registrations/{id}") Object detail(@PathVariable String id) {
46     return s.detail(id);
47 }
48 @PostMapping("/api/staff/registrations/{id}/review") Object review(
49     @PathVariable String id,@RequestBody Map<String,String> b,
50     @RequestHeader("Idempotency-Key") String key,Authentication a) {
51     return run(a,"review:"+id,key,b,() -> s.review(id,b,a.getName()));
52 }

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

53  @PostMapping("/api/staff/scans") Object start(@RequestBody Map<String,String> b,
54  @RequestHeader("Idempotency-Key") String key,Authentication a) {
55  return run(a,"scan",key,b,() -> s.startScan(b,a.getName(),admin(a)));
56  }
57  @GetMapping("/api/staff/scans/{id}") Object scan(@PathVariable String id,
58  Authentication a) { return s.scan(id,a.getName()); }
59  @PostMapping("/api/staff/scans/{id}/cancel") Object cancel(@PathVariable String id,
60  @RequestHeader("Idempotency-Key") String key,Authentication a) {

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

61  return run(a,"cancel:"+id,key,Map.of(),() -> s.cancel(id,a.getName()));
62  }
63  @PostMapping("/api/admin/cards") Object enroll(@RequestBody Map<String,String> b,
64  @RequestHeader("Idempotency-Key") String key,Authentication a) {
65  return run(a,"enroll",key,b,() -> s.enroll(b,a.getName()));
66  }
67  @PostMapping("/api/staff/assignments") Object issue(@RequestBody Map<String,String> b,
68  @RequestHeader("Idempotency-Key") String key,Authentication a) {
69  return run(a,"issue",key,b,() -> s.issue(b,a.getName()));
70  }
71  @GetMapping("/api/staff/assignments") Object assignments(
72  @RequestParam(defaultValue="0") int page) { return s.assignments(page); }
73  @PostMapping("/api/staff/assignments/{id}/return") Object returned(
74  @PathVariable String id,@RequestHeader("Idempotency-Key") String key,Authentication a) {
75  return run(a,"return:"+id,key,Map.of(),() -> s.returned(id,a.getName()));
76  }
77  @GetMapping("/api/admin/cards") Object cards() {
78  return s.db.queryForList("SELECT * FROM cards ORDER BY uid LIMIT 500");
79  }
80  @PostMapping("/api/admin/cards/{uid}/state") Object cardState(
81  @PathVariable String uid,@RequestBody Map<String,String> b,
82  @RequestHeader("Idempotency-Key") String key,Authentication a) {
83  return run(a,"card:"+uid,key,b,() -> s.cardState(uid,b,a.getName()));
84  }
85  @GetMapping("/api/admin/users") Object users() {
86  return s.db.queryForList("SELECT username,role,active FROM users ORDER BY username");
87  }
88  @PostMapping("/api/admin/users") Object user(@RequestBody Map<String,String> b,
89  @RequestHeader("Idempotency-Key") String key,Authentication a) {
90  return run(a,"user",key,b,() -> s.user(b,a.getName()));

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

91  }
92  @GetMapping("/api/admin/settings") Object settings() {
93  return s.one("SELECT hours FROM settings WHERE id=1");
94  }
95  @PostMapping("/api/admin/settings") Object settings(@RequestBody Map<String,String> b,
96  @RequestHeader("Idempotency-Key") String key,Authentication a) {
97  return run(a,"settings",key,b,() -> s.settings(b,a.getName()));
98  }
99  @GetMapping("/api/staff/summary") Object summary() { return s.summary(); }
100 @GetMapping("/api/admin/proofs") Object proofs(@RequestParam(defaultValue="0") int page) {
101 return s.proofs(page);
102 }
103 @GetMapping("/api/admin/proofs/{id}") Object proof(@PathVariable String id) {
104 return s.proof(id);
105 }
106 @PostMapping("/api/device/claim") Object claim(
107 @RequestHeader("Idempotency-Key") String key) {

```


JAVA / LINE NUMBERS ARE NOT SOURCE

```

108     return s.command("reader-1", "claim", key, Map.of(), s::claim);
109 }
110 @PostMapping("/api/device/result/{id}") Object result(@PathVariable String id,
111 @RequestBody Map<String, String> b, @RequestHeader("Idempotency-Key") String key) {
112     return s.command("reader-1", "result:" + id, key, b, () -> s.result(id, b));
113 }
114 @ExceptionHandler(ResponseStatusException.class)
115 ResponseEntity<?> rule(ResponseStatusException e) {
116     return ResponseEntity.status(e.getStatusCode()).body(Map.of(
117         "code", Objects.requireNonNullElse(e.getReason(), "REQUEST_FAILED"),
118         "correlationId", Store.id()));
119 }
120 @ExceptionHandler(DataIntegrityViolationException.class)

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

121 ResponseEntity<?> conflict(DataIntegrityViolationException e) {
122     return ResponseEntity.status(409).body(Map.of("code", "DATA_CONFLICT"));
123 }
124 @ExceptionHandler(Exception.class)
125 ResponseEntity<?> unexpected(Exception e) {
126     String correlation = Store.id();
127     org.slf4j.LoggerFactory.getLogger(Api.class).error("Request {} failed: {}",
128         correlation, e.getClass().getSimpleName());
129     return ResponseEntity.status(500).body(Map.of("code", "INTERNAL_ERROR",
130         "correlationId", correlation));
131 }
132 }

```

Meaning: the controller supplies trusted actor identity and stable operation names to the transaction service.

Purpose: no client actorId can impersonate a staff member. Importance: Critical. Parameterized SQL protects values; all dynamically selected table names are absent here.

Reference limits are explicit: inventory returns at most 500 rows; queue, assignments and proofs have pages of 20. Add cursor/search endpoints before exceeding that inventory scale. The public configuration accepts free-text destinations rather than claiming a hospital directory is already integrated.

11 Frontend build files and request client

frontend/package.json

JSON / LINE NUMBERS ARE NOT SOURCE

```

1 {
2   "name": "hsaas-reference-ui", "private": true, "version": "1.0.0",
3   "type": "module",
4   "scripts": { "dev": "vite", "build": "tsc --noEmit && vite build", "preview": "vite preview" },
5   "dependencies": { "react": "19.1.1", "react-dom": "19.1.1", "qrcode.react": "4.2.0" },
6   "devDependencies": { "@types/react": "19.1.13", "@types/react-dom": "19.1.9",
7     "@vitejs/plugin-react": "5.0.3", "typescript": "5.9.2", "vite": "7.1.5" }
8 }

```

frontend/tsconfig.json

JSON / LINE NUMBERS ARE NOT SOURCE

```

1 {
2   "compilerOptions": {
3     "target": "ES2022", "lib": [ "ES2022", "DOM", "DOM.Iterable" ],
4     "module": "ESNext", "moduleResolution": "Bundler", "jsx": "react-jsx",
5     "strict": true, "skipLibCheck": true, "esModuleInterop": true, "noEmit": true
6   },
7   "include": [ "src" ]
8 }

```

frontend/vite.config.ts

TYPESCRIPT / LINE NUMBERS ARE NOT SOURCE

```

1 import { defineConfig } from 'vite';
2 import react from '@vitejs/plugin-react';
3 export default defineConfig({
4   plugins: [react()],
5   server: { proxy: { '/api': { target: 'http://localhost:8080', changeOrigin: true } } }
6 });

```

frontend/index.html

HTML / LINE NUMBERS ARE NOT SOURCE

```

1 <!doctype html>
2 <html lang="en"><head><meta charset="UTF-8"/>
3 <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
4 <title>HSAAS Visitor and Pass Management</title></head>
5 <body><div id="root"></div><script type="module" src="/src/main.tsx"></script></body>
6 </html>

```

frontend/src/api.ts

TYPESCRIPT / LINE NUMBERS ARE NOT SOURCE

```

1 let csrf:{headerName:string;token:string}|null=null;
2 const pending=new Map<string,string>();
3 export async function token() {
4   const r=await fetch('/api/auth/csrf',{credentials:'include',cache:'no-store'});
5   if (!r.ok) throw new Error('CSRF_UNAVAILABLE');
6   csrf=await r.json(); return csrf!;
7 }
8 export async function api(path:string,body?:Record<string,string>) {
9   const headers:Record<string,string>={};
10  const signature=path+JSON.stringify(body);
11  if (body) {
12    const t=csrf ?? await token(); headers[t.headerName]=t.token;
13    headers['Content-Type']='application/json';
14    // [U01] A transport retry keeps the original command key.
15    if (!pending.has(signature)) pending.set(signature,crypto.randomUUID());
16    headers['Idempotency-Key']=pending.get(signature)!;
17  }
18  const r=await fetch('/api'+path,{method:body?'POST':'GET',headers,
19    credentials:'include',cache:'no-store',body:body?JSON.stringify(body):undefined});
20  const data=r.status===204 ? {} : await r.json();
21  if (!r.ok) {
22    if (r.status===403) csrf=null;
23    if (r.status===401) window.dispatchEvent(new Event('session-expired'));
24    // A received rejection may be retried as a new command after correction.
25    pending.delete(signature); throw new Error(data.code ?? `HTTP_${r.status}`);
26  }
27  pending.delete(signature); return data;
28 }
29 export async function login(username:string,password:string) {
30   const t=await token();

```

TYPESCRIPT / LINE NUMBERS ARE NOT SOURCE

```

31   const r=await fetch('/api/auth/login',{method:'POST',credentials:'include',
32     headers:[{t.headerName}:t.token],body:new URLSearchParams({username,password})});
33   if (!r.ok) throw new Error('LOGIN_FAILED');
34   await token(); return api('/auth/me');
35 }
36 export async function logout() {
37   await api('/auth/logout',{}); csrf=null; pending.clear(); await token();
38 }

```

[U01] Critical: fetch resolving does not mean success. This helper checks HTTP status and keeps the idempotency key only when the outcome is unknown because transport failed. It stores no patient drafts or tokens in localStorage. A full reload loses the in-memory pending key; the teaching UI warns against reloading during an unknown submission. A production recovery flow should provide authenticated or session-bound result lookup.

12 Shared controls and square design tokens

The small Button below is project-owned, shadcn-style semantic composition, not a copied paid component or the complete shadcn distribution. It keeps the reference self-contained. To use actual shadcn/ui generated components, follow the English guide's installation chapter and replace this primitive while retaining its props and accessibility behavior. The application itself does not depend on the CLI.

frontend/src/components/ui/button.tsx

TSX / LINE NUMBERS ARE NOT SOURCE

```
1 import type { ButtonHTMLAttributes } from 'react';
2 export function Button({className='',...props}:ButtonHTMLAttributes<HTMLButtonElement>) {
3   return <button className={'button '+className} {...props}/>;
4 }
```

frontend/src/main.tsx

TSX / LINE NUMBERS ARE NOT SOURCE

```
1 import { createRoot } from 'react-dom/client';
2 import App from './App';
3 import './index.css';
4 createRoot(document.getElementById('root')!).render(<App/>);
```

frontend/src/index.css

CSS / LINE NUMBERS ARE NOT SOURCE

```
1 :root{font-family:Arial,sans-serif;color:#20252b;background:#fafafa;
2   --primary:#9d0b0f;--line:#dce0e4;--radius:0}
3 *{box-sizing:border-box}body{margin:0}a{color:var(--primary)}
4 header{background:white;border-bottom:3px solid var(--primary);padding:20px 28px;
5   display:flex;align-items:center;justify-content:space-between;gap:16px}
6 header strong{font-size:22px}header small{display:block;color:#65717e;margin-top:5px}
7 main{max-width:1250px;margin:0 auto;padding:24px}.public{max-width:640px}
8 nav{display:flex;gap:6px;flex-wrap:wrap;margin-bottom:24px}
9 .button{border:1px solid var(--line);border-radius:0;min-height:44px;
10  padding:10px 15px;background:white;color:#20252b;font:inherit;cursor:pointer}
11 .button.primary,.button[aria-pressed=true]{background:var(--primary);color:white;
12   border-color:var(--primary)}.button.disabled{opacity:.5;cursor:not-allowed}
13 .button:focus-visible,input:focus-visible,select:focus-visible,a:focus-visible{
14   outline:3px solid #1d4ed8;outline-offset:3px}
15 form{display:grid;gap:16px}label{font-weight:bold;display:grid;gap:7px}
16 input,select{font:inherit;border:1px solid #abb3bb;border-radius:0;padding:11px;
17   min-height:44px;width:100%;background:white;color:#20252b}
18 input[type=checkbox]{width:22px;min-height:22px}.check{display:flex;align-items:center}
19 section{background:white;border:1px solid var(--line);padding:24px;margin:18px 0}
20 h1{font-size:27px;margin:0 0 10px}h2{font-size:19px}p{line-height:1.55}
21 .notice{padding:14px;border-left:4px solid var(--primary);background:#fff0f0}
22 .status{padding:12px;background:#edf4ff;border-left:4px solid #2563eb;white-space:pre-wrap}
23 .table-wrap{overflow:auto}table{border-collapse:collapse;width:100%;font-size:14px}
24 th,td{padding:12px;text-align:left;border-bottom:1px solid var(--line);vertical-align:top}
25 th{background:#f4f5f6}td{max-width:270px;overflow-wrap:anywhere}
26 .actions{display:flex;gap:7px;flex-wrap:wrap}.muted{color:#65717e;font-size:14px}
27 pre{white-space:pre-wrap;overflow-wrap:anywhere;background:#f4f5f6;padding:16px}
28 dl{display:grid;grid-template-columns:140px 1fr;gap:10px}dt{font-weight:bold}
29 dd{margin:0;overflow-wrap:anywhere}.scan{border:2px solid var(--primary)}
30 @media(max-width:650px){header{padding:16px;align-items:flex-start}main{padding:16px}}
```

CSS / LINE NUMBERS ARE NOT SOURCE

```
31 section{padding:16px}nav .button{flex:1}dl{grid-template-columns:1fr}}
32 @media print{nav,.actions,header button{display:none}}
```

Importance: High. Square containers and restrained red actions honor the latest design direction. Labels, visible focus, live status and responsive overflow matter more than visual decoration. Test keyboard-only operation and mobile form completion; native alert/confirm dialogs below remain deliberately simple and can later be replaced by accessible shadow Dialog components.

13 Frontend: registration and scanner

frontend/src/App.tsx

This file continues in chapter 14. Concatenate both listings.

TSX / LINE NUMBERS ARE NOT SOURCE

```

1 import { useEffect,useRef,useState } from 'react';
2 import { QRCodeSVG } from 'qrcode.react';
3 import { api,login,logout } from './api';
4 import { Button } from './components/ui/button';
5 type Row=Record<string,any>;
6 const categories=[ 'EXECUTIVE', 'PENJAGA', 'VENDOR', 'CONTRACTOR' ];
7 const sleep=(ms:number)=>new Promise(r=>setTimeout(r,ms));
8 function Fields({names,values,set}: {names:string[];values:Record<string,string>;
9   set:(v:Record<string,string>=>void)} {
10  return <>{names.map(name=><label key={name}>{name}
11    <input name={name} required maxLength={name==='purpose'?200:120}
12      type={name==='password'?'password':name==='phone'?'tel':'text'}
13      value={values[name]??''} onChange={e=>set({...values,[name]:e.target.value})}/>
14    </label>)}</>;
15 }
16 function Registration() {
17  const [form,setForm]=useState<Record<string,string>>({category:'EXECUTIVE'});
18  const [message,setMessage]=useState(''); const [busy,setBusy]=useState(false);
19  const [reference,setReference]=useState('');
20  const set=(v:Record<string,string>=>{setForm(v);setMessage('')});
21  async function submit(e:React.FormEvent) {
22    e.preventDefault();setBusy(true);setMessage('Submitting. Do not reload. ');
23    try {const r=await api('/public/registrations',form);setReference(r.reference);}
24    catch(e){setMessage(String(e)+'\nYour input is preserved. Retry without reloading. ');}
25    finally{setBusy(false);}
26  }
27  return <main className="public"><h1>Visitor registration</h1>
28    <p className="notice">Teaching environment. Hospital validation is MOCK.
29      Use synthetic details only. No door access is granted by this form.</p>
30    {reference?<section><h2>Submitted</h2><p>Show this reference at the counter:</p>

```

TSX / LINE NUMBERS ARE NOT SOURCE

```

31    <strong>{reference}</strong><p>Wait for staff verification and card assignment.</p>
32    <a href="/register">Start another registration</a></section>:
33  <form onSubmit={submit}><fieldset disabled={busy}><legend>Visitor details</legend>
34    <label>Category<select value={form.category}
35      onChange={e=>set({...form,category:e.target.value,mrn:'',ward:'',organization:''})}>
36      {categories.map(c=><option key={c}>{c}</option>)}</select></label>
37    <Fields names={['name','phone','destination','purpose']} values={form} set={set}/>
38    {[ 'VENDOR', 'CONTRACTOR' ].includes(form.category) &&
39      <Fields names={['organization']} values={form} set={set}/>
40    {form.category==='PENJAGA' && <
41      <Fields names={['mrn','ward']} values={form} set={set}/>
42    <p className="muted">Fixture: DEMO-0001 / WARD-A. Timeout: DEMO-TIMEOUT.</p>
43    <Button type="button" onClick={async()}=>{setBusy(true);try{
44      const r=await api('/public/mrn',{mrn:form.mrn?:'',ward:form.ward?''});
45      setMessage('MOCK validation: '+r.result);
46    }catch(e){setMessage(String(e));}finally{setBusy(false);}}>Check MRN</Button>
47  </>
48  <label className="check"><input type="checkbox" required checked={form.consent==='yes'}
49    onChange={e=>set({...form,consent:e.target.checked?'yes':'no'})}/>
50    I understand this demonstration stores the supplied test details.</label>

```

TSX / LINE NUMBERS ARE NOT SOURCE

```

51   <Button className="primary" type="submit">{busy?'Please wait':'Submit registration'}</Button>
52   </fieldset></form>
53   <p role="status" aria-live="polite" className="status">{message||'Ready'}</p>
54   </main>;
55 }
56 function Scan({target,onDone}:{target?:string;onDone:()=>void}) {
57   const [state,setState]=useState('Idle');const [job,setJob]=useState('');
58   const [uid,setUid]=useState('');const [category,setCategory]=useState(categories[0]);
59   const [busy,setBusy]=useState(false);const generation=useRef(0);const current=useRef('');
60   useEffect(()=>())=>{generation.current++;if(current.current)

```

TSX / LINE NUMBERS ARE NOT SOURCE

```

61   void api(`/staff/scans/${current.current}/cancel`,{}).catch(()=>{});,[]);
62   async function cancel() {
63     generation.current++;const old=current.current;current.current='';
64     setJob('');setUid('');setState('Cancelled');setBusy(false);
65     if(old)try{await api(`/staff/scans/${old}/cancel`,{});}catch(e){setState(String(e));}
66   }
67   async function start() {
68     if(current.current) await cancel();
69     const epoch=++generation.current;setBusy(true);setUid('');setState('Creating scan job');
70     try {
71       const j=await api(`/staff/scans',{purpose:target?'ISSUE':'ENROLL',target:target??''});
72       if(epoch!==generation.current){await api(`/staff/scans/${j.id}/cancel`,{});return;}
73       current.current=j.id;setJob(j.id);setState('Remove any card, then tap the selected card');
74       for(let n=0;n<95;n++) {
75         await sleep(1000);if(epoch!==generation.current)return;
76         const r=await api(`/staff/scans/${j.id}`);
77         // [U02] Late results from a cancelled scan cannot update this panel.
78         if(epoch!==generation.current)return;
79         if(r.state==='READY'){setUid(r.uid);setState('Read complete. Confirm below.');
```

```

91     setState(target?'Assigned ${r.uid}. Hand card to visitor. Proof PENDING.': 'Card enrolled.');
```

TSX / LINE NUMBERS ARE NOT SOURCE

```

92     onDone();
93   }catch(e){setState(String(e));}finally{setBusy(false);}
94 }
95 return <section className="scan"><h2>{target?'Assign access card':'Enroll access card'}</h2>
96 <p className="muted">Category source: inventory mapping. One configured counter reader.</p>
97 {!target&&<label>Inventory category<select disabled={busy} value={category}
98   onChange={e=>setCategory(e.target.value)}>{categories.map(c=><option key={c}>{c}</option>)}
99 </select></label>}
100 <p className="status" role="status">{state}</p><uid&&<p>Scanned UID: <strong>{uid}</strong></p>
101 <div className="actions"><Button disabled={busy} onClick={start}>Scan / retry</Button>
102   <Button disabled={!job} onClick={cancel}>Cancel</Button>
103   <Button className="primary" disabled={!uid||busy} onClick={commit}>Confirm</Button></div>
104 </section>;
105 }

```

TSX / LINE NUMBERS ARE NOT SOURCE

```
106 function Table({rows,action}:{rows:Row[];action?:(r:Row)=>React.ReactNode}) {
107   const keys=rows.length?Object.keys(rows[0]):[];
108   return rows.length?<div className="table-wrap"><table><thead><tr>
109     {keys.map(k=><th key={k}>{k}</th>)}{action&&<th>Actions</th></tr></thead>
110     <tbody>{rows.map((r,i)=><tr key={r.id??r.uid??r.username??i}>
111       {keys.map(k=><td key={k}>{r[k]==null?'--':String(r[k])}</td>)}
112       {action&&<td>{action(r)}</td></tr>}}</tbody></table></div>:<p>No records found.</p>;
113 }
```

[U02] High: generation counters prevent stale scan results from changing a newly opened context. Cancel also calls Backend; local cancellation alone cannot revoke server evidence. The scan panel is remounted when the selected registration changes.

The server repeats MRN validation regardless of whether the separate Check MRN button was used. The registration form preserves input on errors but deliberately avoids persistent browser storage of patient details.

14 Frontend: staff and administrator workspace

App.tsx, final continuation

TSX / LINE NUMBERS ARE NOT SOURCE

```

1 export default function App() {
2   const [user, setUser] = useState<Row|null>(null); const [ready, setReady] = useState(false);
3   const [credentials, setCredentials] = useState<Record<string, string>>({});
4   const [tab, setTab] = useState('Queue'); const [page, setPage] = useState(0);
5   const [rows, setRows] = useState<Row[]>([]); const [detail, setDetail] = useState<Row|null>(null);
6   const [summary, setSummary] = useState<Row>({}); const [message, setMessage] = useState('');
7   const [busy, setBusy] = useState(false); const [revision, setRevision] = useState(0);
8   const [account, setAccount] = useState<Record<string, string>>({
9     role: 'COUNTER_STAFF', active: 'true' });
10  const [hours, setHours] = useState('8');
11  const refresh = () => setRevision(x => x + 1);
12  const publicPage = window.location.pathname === '/register';
13  useEffect(() => { api('/auth/me').then(setUser).catch(() => {}).finally(() => setReady(true));
14    const end = () => { setUser(null); setDetail(null); };
15    window.addEventListener('session-expired', end);
16    return () => window.removeEventListener('session-expired', end); }, []);
17  useEffect(() => {
18    if (!user || publicPage) return;
19    let live = true;
20    async function load() { try {
21      const paths: Record<string, string> = { Queue: '/staff/registrations',
22        Assignments: '/staff/assignments', Inventory: '/admin/cards',
23        Users: '/admin/users', Audit: '/admin/proofs' };
24      if (paths[tab]) { const r = await api(paths[tab] + `?page=${page}`); if (live) setRows(r); }
25      if (tab === 'Summary') { const r = await api('/staff/summary'); if (live) setSummary(r); }
26      if (tab === 'Settings') { const r = await api('/admin/settings'); if (live) setHours(String(r.hours)); }
27    } catch (e) { if (live) setMessage(String(e)); } }
28    void load(); const timer = ['Queue', 'Assignments', 'Inventory', 'Audit', 'Summary'].includes(tab)
29      ? setInterval(load, 5000) : undefined;
30    return () => { live = false; if (timer) clearInterval(timer); };

```

TSX / LINE NUMBERS ARE NOT SOURCE

```

31   }, [tab, page, user, revision, publicPage]);
32   async function action(fn: () => Promise<unknown>) {
33     setBusy(true); setMessage('Working...');
34     try { await fn(); setMessage('Completed'); refresh(); }
35     catch (e) { setMessage(String(e)); } finally { setBusy(false); }
36   }
37   const header = <header><div><strong>HSAAS</strong><small>Visitor & Pass Management</small></div>
38     <div><user><span>{user.username} / {user.admin ? 'Admin': 'Counter staff'} </span>
39     <Button onClick={() => action(async () => { await logout(); setUser(null); })}>Log out</Button></div>
40     <a href={publicPage ? '/' : '/register'}>{publicPage ? 'Staff login': 'Visitor registration'}</a></div>
41   </header>;
42   if (publicPage) return <>{header}<Registration></></>;
43   if (!ready) return <>{header}<main><p>Loading session...</p></main></>;
44   if (!login) return <>{header}<main class="public"><h1>Staff login</h1>
45     <form onSubmit={e => { e.preventDefault(); void action(async () =>
46       setUser(await login(credentials.username ?? '', credentials.password ?? '')); }}>
47       <Fields names={['username', 'password']} values={credentials} set={setCredentials}/>
48       <Button className="primary" disabled={busy}>Log in</Button></form>
49     <p role="alert">{message}</p></main></>;
50   const tabs = ['Queue', 'Assignments', 'Summary', 'QR',
51     ...(user.admin ? ['Inventory', 'Users', 'Settings', 'Audit']: [])];
52   return <>{header}<main><nav aria-label="Workspace">{tabs.map(t =>

```

TSX / LINE NUMBERS ARE NOT SOURCE

```

53   <Button key={t} aria-pressed={tab===t} onClick={()=>{
54     setTab(t);setPage(0);setRows([]);setDetail(null);setMessage('');}}>{t}</Button>}}</nav>
55   <h1>{tab}</h1><p className="notice">MOCK hospital adapter. Synthetic data only.
56     Card category is assigned in inventory. Proof confirmation is asynchronous.</p>
57   <p className="status" role="status">{message||'Ready'}</p>
58   [{ 'Queue', 'Assignments', 'Audit' }.includes(tab)&&<div className="actions">
59     <Button disabled={!page} onClick={()=>setPage(x=>x-1)}>Previous</Button>
60     <span>Page {page+1}</span><Button disabled={rows.length<20}

```

TSX / LINE NUMBERS ARE NOT SOURCE

```

61     onClick={()=>setPage(x=>x+1)}>Next</Button></div>}
62   {tab==='Queue'&&<Table rows={rows} action={r=><Button disabled={busy}
63     onClick={()=>action(async()=>setDetail(await api(`/staff/registrations/${r.id}`)))>
64     Review</Button>}>{detail&&<section><h2>Registration {detail.id}</h2>
65     <dl>{Object.entries(detail.data).map(([k,v])=><div key={k}><dt>{k}</dt>
66       <dd>{String(v)}</dd></div>)}</dl><p>{detail.category} / {detail.state}</p>
67     {detail.state==='SUBMITTED'&&<div className="actions">
68       <Button disabled={busy} onClick={()=>action(async()=>{
69         await api(`/staff/registrations/${detail.id}/review`,{decision:'VERIFIED'});
70         setDetail(await api(`/staff/registrations/${detail.id}`));}}>Verify information</Button>
71       <Button disabled={busy} onClick={()=>{const reason=prompt('Reason for rejection');
72         if(reason)void action(async()=>{await api(`/staff/registrations/${detail.id}/review`,
73           {decision:'REJECTED',reason});setDetail(null);}}}>Reject</Button></div>
74       {detail.state==='VERIFIED'&&<Scan key={detail.id} target={detail.id} onDone={refresh}/>}
75     </section></div>}
76   {tab==='Assignments'&&<Table rows={rows} action={r=>r.state!=='RETURNED'&&
77     <Button disabled={busy} onClick={()=>{if(confirm(`Confirm return of ${r.uid}`?`))
78       void action(()=>api(`/staff/assignments/${r.id}/return`,{ }));}}>Return</Button>}}>
79   {tab==='Inventory'&&<Scan onDone={refresh}/><p>First 500 cards, ordered by UID.</p>
80   <Table rows={rows} action={r=><Button disabled={busy||r.state==='ISSUED'}
81     onClick={()=>action(()=>api(`/admin/cards/${r.uid}/state`,
82       {state:r.state==='DISABLED'? 'AVAILABLE': 'DISABLED'})}>
83     {r.state==='DISABLED'? 'Enable': 'Disable'}</Button>}}></div>
84   {tab==='Users'&&<Table rows={rows} action={r=><Button onClick={()=>setAccount(
85     {username:r.username,role:r.role,active:String(r.active),password:''})}>Edit</Button>}}>
86   <section><h2>Create or edit account</h2><form onSubmit={e=>{e.preventDefault();
87     void action(()=>api(`/admin/users`,account));}}>
88   <Fields names={['username']} values={account} set={setAccount}/>
89   <label>New account password only (16+ characters)
90   <input type="password" maxLength={72} value={account.password??''}

```

TSX / LINE NUMBERS ARE NOT SOURCE

```

91     onChange={e=>setAccount({...account,password:e.target.value})}></label>
92   <label>Role<select value={account.role} onChange={e=>setAccount({...account,role:e.target.value})}>
93     <option>COUNTER_STAFF</option><option>ADMIN</option></select></label>
94   <label>Active<select value={account.active}
95     onChange={e=>setAccount({...account,active:e.target.value})}>
96     <option value="true">Enabled</option><option value="false">Disabled</option></select></label>
97   <Button disabled={busy}>Save account</Button></form></section></div>
98   {tab==='Settings'&&<form onSubmit={e=>{e.preventDefault();
99     void action(()=>api(`/admin/settings`,{hours}));}}><label>Loan duration (hours)
100   <input type="number" min="1" max="168" value={hours}
101     onChange={e=>setHours(e.target.value)}></label><Button disabled={busy}>Save</Button>
102   <p>Changes affect future assignments only.</p></form>
103   {tab==='Summary'&&<section><p>Pending review: {summary.queue} / Active: {summary.active}
104     { ' / ' }Overdue: {summary.overdue}</p><h2>All-time registrations by category</h2>
105     <Table rows={summary.registrations??[]}></section>
106   {tab==='QR'&&<section><h2>Scan to register</h2>
107     <QRCodeSVG value={window.location.origin+'/register'} size={240} marginSize={4}/>

```

TSX / LINE NUMBERS ARE NOT SOURCE

```

108     <p>{window.location.origin}/register</p>
109     <p>Use the deployed HTTPS address, not localhost, when printing for phones.</p></section>
110     {tab==='Audit'&&<><Table rows={rows} action={r=><Button onClick={()=>action(async()=>{
111       const p=await api(`/admin/proofs/${r.id}`);setDetail(p);}}>Inspect</Button>}/>
112       {detail&&<section><h2>Local snapshot check</h2><p>
113         {detail.localIntegrity?'LOCAL MATCH':'LOCAL MISMATCH'} - not an on-chain verdict.</p>
114         <pre>{detail.payload}</pre><p>Independent verification:
115         {' node verify.mjs '+detail.id}</p></section>}</>
116     </main></>;
117   }

```

Review, assignment, return and admin commands are complete UI interactions, not simulated success messages. Every mutation awaits Backend. Polling updates operational state but does not replace server authorization. The verifier is a privileged command-line tool instead of embedding a private database connection in the browser.

Known UI boundaries: native confirmations instead of custom dialogs, English-only copy, no historical chart/export builder and no approved hospital branding. Use the guideline's design process to refine these without weakening the backend invariants.

15 Windows reader agent

reader-agent/pom.xml

XML / LINE NUMBERS ARE NOT SOURCE

```

1 <project xmlns="http://maven.apache.org/POM/4.0.0">
2   <modelVersion>4.0.0</modelVersion>
3   <groupId>edu.upm</groupId><artifactId>reader-agent</artifactId><version>1.0.0</version>
4   <properties><maven.compiler.release>21</maven.compiler.release>
5     <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding></properties>
6   <dependencies><dependency><groupId>com.fasterxml.jackson.core</groupId>
7     <artifactId>jackson-databind</artifactId><version>2.19.2</version></dependency></dependencies>
8   <build><plugins><plugin><groupId>org.apache.maven.plugins</groupId>
9     <artifactId>maven-compiler-plugin</artifactId><version>3.14.0</version></plugin>
10    <plugin><groupId>org.codehaus.mojo</groupId><artifactId>exec-maven-plugin</artifactId>
11      <version>3.5.0</version><configuration><mainClass>ReaderAgent</mainClass></configuration>
12    </plugin></plugins></build>
13 </project>

```

reader-agent/src/main/java/ReaderAgent.java

JAVA / LINE NUMBERS ARE NOT SOURCE

```

1 import com.fasterxml.jackson.databind.ObjectMapper;
2 import com.fasterxml.jackson.core.type.TypeReference;
3 import javax.smartcardio.*;
4 import java.net.URI;
5 import java.net.http.*;
6 import java.time.Duration;
7 import java.util.*;
8
9 public class ReaderAgent {
10   static final ObjectMapper JSON=new ObjectMapper();
11   static final HttpClient HTTP=HttpClient.newBuilder()
12     .connectTimeout(Duration.ofSeconds(8)).build();
13   static String env(String key) {
14     String v=System.getenv(key);
15     if(v==null||v.isBlank())throw new IllegalStateException("Missing "+key);
16     return v;
17   }
18   static Map<String,Object> post(String path,Map<String,String> body,String key)
19   throws Exception {
20     String base=env("API_URL");
21     URI uri=URI.create(base);
22     if(!uri.getScheme().equals("https") &&
23       !(uri.getScheme().equals("http")&&Set.of("localhost","127.0.0.1").contains(uri.getHost())))
24       throw new IllegalStateException("HTTPS required outside loopback");
25     var request=HttpRequest.newBuilder(URI.create(base+"/api/device"+path))
26       .timeout(Duration.ofSeconds(15)).header("Authorization","Bearer "+env("DEVICE_TOKEN"))
27       .header("Content-Type","application/json").header("Idempotency-Key",key)
28       .POST(HttpRequest.BodyPublishers.ofString(JSON.writeValueAsString(body))).build();
29     Exception last=null;
30     for(int attempt=0;attempt<3;attempt++) {

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

31     try {
32         var r=HTTP.send(request,HttpResponse.BodyHandlers.ofString());
33         if(r.statusCode()!=200)throw new IllegalStateException("DEVICE_HTTP_"+r.statusCode());
34         return JSON.readValue(r.body(),new TypeReference<Map<String,Object>>(){});
35     }catch(java.io.IOException e){last=e;Thread.sleep(1000L*(attempt+1));}
36     }
37     throw last;
38 }
39 static String read() throws Exception {
40     String mock=System.getenv("MOCK_UID");
41     if(mock!=null&&!mock.isBlank()) {
42         System.out.println("MOCK READER: simulated tap, no physical evidence");
43         Thread.sleep(1500);return mock;
44     }
45     var terminals=TerminalFactory.getDefault().terminals().list();
46     String name=env("READER_NAME");
47     var terminal=terminals.stream().filter(t->t.getName().equals(name))
48     .findFirst().orElseThrow(()->new IllegalStateException("READER_NOT_FOUND"));
49     // [R02] Removal gates successive scans; one held card is not many taps.
50     if(terminal.isCardPresent())&&!terminal.waitForCardAbsent(15000))
51         throw new IllegalStateException("REMOVE_CARD");
52     System.out.println("Tap card now");
53     if(!terminal.waitForCardPresent(45000))throw new IllegalStateException("READ_TIMEOUT");
54     Card card=terminal.connect("*");
55     try {
56         var response=card.getBasicChannel().transmit(new CommandAPDU(new byte[] {
57             (byte)0xff,(byte)0xca,0,0,0});
58         if(response.getSW()!=0x9000)throw new IllegalStateException("APDU_STATUS");
59         byte[] bytes=response.getData();
60         if(bytes.length<4||bytes.length>10)throw new IllegalStateException("UID_LENGTH");

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

61         return HexFormat.of().withUpperCase().formatHex(bytes);
62     }finally{card.disconnect(false);}
63     }
64     public static void main(String[] args) throws Exception {
65         if(args.length>0&&args[0].equals("list")) {
66             TerminalFactory.getDefault().terminals().list().forEach(t->System.out.println(t.getName()));
67             return;
68         }
69         env("API_URL");env("DEVICE_TOKEN");
70         while(!Thread.currentThread().isInterrupted()) {
71             try {
72                 var job=post("/claim",Map.of(),UUID.randomUUID().toString());
73                 if("READING".equals(job.get("state"))) {
74                     Map<String,String> result=new HashMap<>();
75                     result.put("lease",job.get("lease").toString());
76                     try {result.put("uid",read());result.put("state","READY");}
77                     catch(Exception e){result.put("state","ERROR");System.err.println("Read failed");}
78                     post("/result/"+job.get("id"),result,UUID.randomUUID().toString());
79                 }
80             }catch(Exception e){System.err.println("Agent operation failed: "+e.getClass().getSimpleName());}
81             Thread.sleep(2000);
82         }
83     }
84 }

```

[R02] High: without removal gating, leaving a card on the reader can satisfy a later job unintentionally. A failed read uploads ERROR, never the last successful UID. HTTPS protects bearer-token transit. This single-device reference requires a stable API_URL without a trailing slash and the exact READER_NAME shown by list

mode.

Windows setup: install the vendor driver; verify Smart Card service and the contactless slot; run `mvn compile exec:java -Dexec.args=list` to enumerate; then set `API_URL`, `DEVICE_TOKEN` and `READER_NAME`. Use `MOCK_UID` only for explicitly labelled mock tests. Never mix mock claims with real evidence. Set Backend `READER_LABEL` to 'MOCK' when testing that mode.

For autostart, first verify native Windows execution, then package dependencies or use a wrapper script launched by Task Scheduler under a restricted account. Do not run the reader in Railway or assume WSL USB access. Secrets belong in protected machine configuration; rotate `DEVICE_TOKEN` on both ends when compromised.

16 Move package and immutable commitments

move/hsaas_audit/Move.toml

TOML / LINE NUMBERS ARE NOT SOURCE

```
1 [package]
2 name = "hsaas_audit"
3 edition = "2024"
4
5 [addresses]
6 hsaas_audit = "0x0"
```

Use a current Testnet-compatible Sui CLI with automatic framework dependencies. If its generated manifest differs, start from `sui move new hsaas_audit` and retain its framework resolution rather than guessing a Git revision. Commit the generated `Move.lock` after build. The sources below target the 2024 edition.

move/hsaas_audit/sources/audit.move

MOVE / LINE NUMBERS ARE NOT SOURCE

```
1 module hsaas_audit::audit {
2   use sui::object::{Self, UID, ID};
3   use sui::table::{Self, Table};
4   use sui::transfer;
5   use sui::event;
6   use sui::tx_context::{Self, TxContext};
7
8   public struct WriterCap has key, store { id: UID, registry: ID }
9   public struct Registry has key {
10     id: UID,
11     proofs: Table<vector<u8>, vector<u8>>,
12   }
13   public struct Anchored has copy, drop {
14     registry: ID, event_id: vector<u8>, hash: vector<u8>, version: u64,
15   }
16   fun create(ctx: &mut TxContext): (WriterCap, Registry) {
17     let registry=Registry { id: object::new(ctx), proofs: table::new(ctx) };
18     let cap=WriterCap { id: object::new(ctx), registry: object::id(&registry) };
19     (cap, registry)
20   }
21   fun init(ctx: &mut TxContext) {
22     let (cap, registry)=create(ctx);
23     transfer::public_transfer(cap,tx_context::sender(ctx));
24     transfer::share_object(registry);
25   }
26   public entry fun anchor(cap: &WriterCap, registry: &mut Registry,
27     event_id: vector<u8>, hash: vector<u8>, version: u64) {
28     // [M01] Capability is bound to this registry, not merely to a type.
29     assert!(cap.registry==object::id(registry),1);
30     assert!(event_id.length()==36 && hash.length()==32 && version==1,2);
```

MOVE / LINE NUMBERS ARE NOT SOURCE

```
31   if (table::contains(&registry.proofs,&event_id)) {
32     // [M02] An existing event can never be overwritten with a new hash.
33     assert!(*table::borrow(&registry.proofs,&event_id)==hash,3);
34   } else {
35     table::add(&mut registry.proofs,copy event_id,copy hash);
36   };
```

MOVE / LINE NUMBERS ARE NOT SOURCE

```
37     event::emit(Anchored {registry:object::id(registry),event_id,hash,version});
38 }
39 #[test_only]
40 public fun fixture(ctx: &mut TxContext): (WriterCap, Registry) { create(ctx) }
41 #[test_only]
42 public fun finish(cap: WriterCap, registry: Registry, ctx: &TxContext) {
43     transfer::public_transfer(cap,tx_context::sender(ctx));
44     transfer::share_object(registry);
45 }
46 }
```

[M01] Critical: possession of WriterCap is necessary; a capability created for a different registry cannot write here. Only initialization can create production resources. Do not add an unrestricted public mint-cap function.

[M02] Critical: identical event/hash retries preserve the commitment; a conflicting hash aborts. An identical retry may emit another matching event, but never a conflicting stored proof. The worker normally retries the exact same signed transaction, preserving the original transaction digest and effects.

Retained UpgradeCap authority affects trust assumptions. Keep publisher/upgrade secrets away from the worker and document whether upgrades remain possible. 'Immutable commitment' describes this module's update rules, not a promise that an administrator with upgrade powers can never change application behavior.

17 Move tests and publication procedure

move/hsaas_audit/tests/audit_tests.move

MOVE / LINE NUMBERS ARE NOT SOURCE

```

1  #[test_only]
2  module hsaas_audit::audit_tests {
3      use hsaas_audit::audit;
4      use sui::tx_context;
5
6      #[test]
7      fun identical_retry() {
8          let mut ctx=tx_context::dummy();
9          let (cap, mut registry)=audit::fixture(&mut ctx);
10         let id=b"12345678-1234-1234-1234-123456789012";
11         let hash=b"01234567890123456789012345678901";
12         audit::anchor(&cap,&mut registry,id,hash,1);
13         audit::anchor(&cap,&mut registry,id,hash,1);
14         audit::finish(cap,registry,&ctx);
15     }
16     #[test]
17     #[expected_failure(abort_code=3, location=hsaas_audit::audit)]
18     fun conflicting_hash() {
19         let mut ctx=tx_context::dummy();
20         let (cap, mut registry)=audit::fixture(&mut ctx);
21         let id=b"12345678-1234-1234-1234-123456789012";
22         audit::anchor(&cap,&mut registry,id,b"01234567890123456789012345678901",1);
23         audit::anchor(&cap,&mut registry,id,b"11234567890123456789012345678901",1);
24         audit::finish(cap,registry,&ctx);
25     }
26     #[test]
27     #[expected_failure(abort_code=2, location=hsaas_audit::audit)]
28     fun invalid_length() {
29         let mut ctx=tx_context::dummy();
30         let (cap, mut registry)=audit::fixture(&mut ctx);

```

MOVE / LINE NUMBERS ARE NOT SOURCE

```

31     audit::anchor(&cap,&mut registry,b"bad",b"short",1);
32     audit::finish(cap,registry,&ctx);
33 }
34 #[test]
35 #[expected_failure(abort_code=1, location=hsaas_audit::audit)]
36 fun wrong_registry() {
37     let mut ctx=tx_context::dummy();
38     let (cap1, reg1)=audit::fixture(&mut ctx);
39     let (cap2, mut reg2)=audit::fixture(&mut ctx);
40     audit::anchor(&cap1,&mut reg2,b"12345678-1234-1234-1234-123456789012",
41         b"01234567890123456789012345678901",1);
42     audit::finish(cap1,reg1,&ctx); audit::finish(cap2,reg2,&ctx);
43 }
44 }

```

Run from the package directory using Bash/WSL or the supported native CLI:

BASH / LINE NUMBERS ARE NOT SOURCE

```
1 sui --version
2 sui client switch --env testnet
3 sui client active-address
4 sui client gas
5 sui move build
6 sui move test
7 sui client publish --gas-budget 100000000
```

The gas budget is an example cap in MIST, not a guaranteed price. Obtain Testnet tokens from the official faucet. Save the successful package ID, shared Registry ID, address-owned WriterCap ID, transaction digest and UpgradeCap custody. Transfer the WriterCap deliberately if using a separate worker address, then verify ownership before running the worker.

Acceptance is the actual CLI output from your pinned version plus a real Testnet round trip. None is claimed already executed. A different address attempting to use someone else's owned capability should also fail at transaction authorization; record that negative network test separately.

18 Worker configuration and durable submission

This reference uses a single worker guarded by a MySQL connection-owned named lock, rather than the guideline's multi-worker lease design. The lock is not a business transaction or a row lock. Signed bytes and signature are persisted before submission, so an interrupted worker can replay the exact transaction. Do not run another tool spending the worker's gas objects while an unknown transaction is being reconciled.

sui-worker/package.json

JSON / LINE NUMBERS ARE NOT SOURCE

```

1 {
2   "name": "hsaas-audit-worker", "private": true, "version": "1.0.0", "type": "module",
3   "scripts": { "start": "node worker.mjs", "verify": "node verify.mjs",
4     "check": "node --check worker.mjs && node --check verify.mjs" },
5   "dependencies": { "@mysten/sui": "^2.0.0", "mysql2": "^3.14.0" }
6 }
```

Install once with `npm install`, inspect resolved SDK version and commit `package-lock.json`. Thereafter use `npm ci`. The listings use the documented v2 client API checked for this edition; if a resolved minor release changes types, consult its API definitions and preserve the lockfile. Worker code is JavaScript ESM so that the PDF does not require a separate compiler configuration.

sui-worker/worker.mjs

JAVASCRIPT / LINE NUMBERS ARE NOT SOURCE

```

1 import mysql from 'mysql2/promise';
2 import { createHash } from 'node:crypto';
3 import { SuiGrpcClient } from '@mysten/sui/grpc';
4 import { Transaction } from '@mysten/sui/transactions';
5 import { Ed25519Keypair } from '@mysten/sui/keypairs/ed25519';
6 import { normalizeSuiAddress } from '@mysten/sui/utis';
7
8 const env=k=>{if(!process.env[k])throw Error('Missing '+k);return process.env[k];};
9 const sleep=ms=>new Promise(r=>setTimeout(r,ms));
10 const db=await mysql.createConnection({host:env('DB_HOST'),port:+env('DB_PORT'),
11   user:env('DB_USER'),password:env('DB_PASSWORD'),database:env('DB_NAME')});
12 const client=new SuiGrpcClient({network:'testnet',baseUrl:env('SUI_GRPC_URL')});
13 const signer=Ed25519Keypair.fromSecretKey(env('SUI_PRIVATE_KEY'));
14 if(normalizeSuiAddress(signer.toSuiAddress())!==normalizeSuiAddress(env('EXPECTED_ADDRESS'))){
15   throw Error('WRONG_SIGNER');
16 }
17 // [W01] Check the chain identity; the network label alone is not verification.
18 if((await client.getChainIdentifier()).chainIdentifier!==env('EXPECTED_CHAIN_ID')){
19   throw Error('WRONG_CHAIN');
20 }
21 const [lock]=await db.query("SELECT GET_LOCK('hsaas-audit-worker',0) AS acquired");
22 if(lock[0].acquired!==1)throw Error('WORKER_ALREADY_RUNNING');
23 const balance=await client.getBalance({owner:signer.toSuiAddress()});
24 if(BigInt(balance.balance.balance)===0n)throw Error('NO_TEST_GAS');
25 const cap=await client.getObject({objectId:env('WRITER_CAP_ID')});
26 if(cap.object.owner.$kind!=='AddressOwner' ||
27   normalizeSuiAddress(cap.object.owner.AddressOwner)!==normalizeSuiAddress(signer.toSuiAddress())){
28   throw Error('CAP_NOT_OWNED');
29 }
30 let stopped=false;
31 process.on('SIGTERM',()=>{stopped=true;});process.on('SIGINT',()=>{stopped=true;});
32 try {
33   while(!stopped) {
```

JAVASCRIPT / LINE NUMBERS ARE NOT SOURCE

```

31  const [rows]=await db.execute(`
32  SELECT o.*,a.hash,a.payload FROM outbox o
33  JOIN audit_events a ON a.id=o.event_id
34  WHERE o.state IN ('PENDING','SUBMITTED','RETRY') AND o.attempts<12
35  ORDER BY a.created_at,a.id LIMIT 1`);
36  if(!rows.length){await sleep(2000);continue;}
37  const row=rows[0];
38  const local=createHash('sha256').update(row.payload,'utf8').digest('hex');
39  if(local!==row.hash){
40    await db.execute("UPDATE outbox SET state='FAILED',error_code='LOCAL_HASH' WHERE event_id=?",
41      [row.event_id]);continue;
42  }
43  let bytes=row.tx_bytes?Buffer.from(row.tx_bytes,'base64'):null;
44  let signature=row.signature;
45  try {
46    if(!bytes) {
47      const tx=new Transaction();tx.setSender(signer.toSuiAddress());
48      tx.moveCall({target:`${env('PACKAGE_ID')}::audit::anchor`,arguments:[
49        tx.object(env('WRITER_CAP_ID')),tx.object(env('REGISTRY_ID')),
50        tx.pure.vector('u8',[...Buffer.from(row.event_id,'utf8')]),
51        tx.pure.vector('u8',[...Buffer.from(row.hash,'hex')]),tx.pure.u64(1)]});
52      bytes=await tx.build({client});
53      ({signature}=await signer.signTransaction(bytes));
54    }
55  }catch(e){
56    await db.execute(`UPDATE outbox SET state='RETRY',attempts=attempts+1,
57      error_code='BUILD_UNAVAILABLE' WHERE event_id=?`,[row.event_id]);
58    await sleep(10000);continue;
59  }
60  // [W02] Commit exact signed intent BEFORE any execution request.

```

JAVASCRIPT / LINE NUMBERS ARE NOT SOURCE

```

61  // A DB failure here exits the worker; unsigned/unpersisted work is not submitted.
62  await db.execute(`UPDATE outbox SET tx_bytes=?,signature=?,state='SUBMITTED',
63    attempts=attempts+1 WHERE event_id=?`,[Buffer.from(bytes).toString('base64'),signature,row.event_id]);
64  let result;
65  try {
66    result=row.digest?await client.getTransaction({digest:row.digest}):
67      await client.executeTransaction({transaction:bytes,signatures:[signature]});
68  }catch(e){
69    await db.execute("UPDATE outbox SET error_code='OUTCOME_UNKNOWN' WHERE event_id=?",
70      [row.event_id]);
71    await sleep(Math.min(60000,2000*2**Math.min(row.attempts,5)));continue;
72  }
73  const transaction=result.Transaction??result.FailedTransaction;
74  await db.execute("UPDATE outbox SET digest=? WHERE event_id=?",
75    [transaction.digest,row.event_id]);
76  if(result.$kind==='FailedTransaction'&&!transaction.status.success){
77    await db.execute("UPDATE outbox SET state='FAILED',error_code='MOVE_FAILED' WHERE event_id=?",
78      [row.event_id]);continue;
79  }
80  try {await client.waitForTransaction({digest:transaction.digest,timeout:30000});}
81  catch(e){await sleep(5000);continue;}
82  // [W03] A known successful, indexed transaction is required for CONFIRMED.
83  await db.execute("UPDATE outbox SET state='CONFIRMED',error_code=NULL WHERE event_id=?",
84    [row.event_id]);
85  }
86  }finally{
87    await db.query("SELECT RELEASE_LOCK('hsaas-audit-worker')");await db.end();
88  }

```

[W01] Critical: an environment label does not stop a misconfigured URL from pointing to another chain. Set EXPECTED_CHAIN_ID from a trusted Testnet reference and verify it once before normal operation. Package/cap/registry must all belong to that same deployment.

[W02] Critical: replaying the same signed bytes prevents an ambiguous network response from creating a new transaction intent. Database query failures are fatal rather than silently continuing without the named lock. MySQL connection loss releases that lock; a restarted worker resumes stored bytes. Test connection loss and process termination around each persistence boundary.

[W03] Critical: a returned FailedTransaction is not success. The worker records definitive failure; unknown network outcome remains SUBMITTED. After twelve attempts, rows stop automatic retry and require operator investigation; they are not labelled successful or definitively failed just because time elapsed. The named lock requires the same MySQL server across workers and a direct persistent connection, not a transaction-pooling proxy.

19 Independent on-chain proof verifier

sui-worker/verify.mjs

This tool needs read-only database access, expected network/package/registry settings and an event ID. It does not need the signing key. It parses the BCS event fields emitted by the exact Move layout in chapter 16, instead of trusting an outbox status label or loosely shaped JSON.

JAVASCRIPT / LINE NUMBERS ARE NOT SOURCE

```

1 import mysql from 'mysql2/promise';
2 import { createHash } from 'node:crypto';
3 import { SuiGrpcClient } from '@mysten/sui/grpc';
4 import { bcs } from '@mysten/sui/bcs';
5 import { normalizeSuiAddress } from '@mysten/sui/utis';
6 const env=k=>{if(!process.env[k])throw Error('Missing '+k);return process.env[k];};
7 const id=process.argv[2];
8 if(!id||!/^[\0-9a-f-]{36}$/i.test(id))throw Error('Supply event UUID');
9 const db=await mysql.createConnection({host:env('DB_HOST'),port:+env('DB_PORT'),
10   user:env('DB_USER'),password:env('DB_PASSWORD'),database:env('DB_NAME')});
11 const eventType=bcs.struct('Anchored',{
12   registry:bcs.Address,event_id:bcs.vector(bcs.u8()),hash:bcs.vector(bcs.u8()),version:bcs.u64()
13 });
14 try{
15   const [rows]=await db.execute(`SELECT a.payload,a.hash,o.digest
16     FROM audit_events a JOIN outbox o ON a.id=o.event_id WHERE a.id=?`,[id]);
17   if(!rows.length)throw Error('LOCAL_EVENT_NOT_FOUND');
18   const row=rows[0];const computed=createHash('sha256').update(row.payload,'utf8').digest('hex');
19   if(computed!==row.hash){console.log('MISMATCH: local payload changed');process.exitCode=2;}
20   else if(!row.digest){console.log('NOT_ANCHORED_OR_UNKNOWN: no recorded transaction digest');}
21   else{
22     const client=new SuiGrpcClient({network:'testnet',baseUrl:env('SUI_GRPC_URL')});
23     if((await client.getChainIdentifier()).chainIdentifier!==env('EXPECTED_CHAIN_ID'))
24       throw Error('WRONG_CHAIN');
25     const result=await client.getTransaction({digest:row.digest,include:{events:true}});
26     if(result.$kind==='FailedTransaction'){
27       console.log('NOT_ANCHORED: recorded transaction failed');process.exitCode=2;
28     }else{
29       const expected=normalizeSuiAddress(env('PACKAGE_ID'))+'::audit::Anchored';
30       const matches=result.Transaction.events.filter(e=>e.eventType===expected)

```

JAVASCRIPT / LINE NUMBERS ARE NOT SOURCE

```

31     .map(e=>eventType.parse(e.bcs)).filter(e=>
32       normalizeSuiAddress(e.registry)===normalizeSuiAddress(env('REGISTRY_ID')) &&
33       Buffer.from(e.event_id.toString('utf8'))===id && String(e.version)===1);
34   const ok=matches.some(e=>Buffer.from(e.hash).toString('hex')===computed);
35   console.log(ok?'MATCH: snapshot matches the on-chain commitment':'MISMATCH: no matching event');
36   if(!ok)process.exitCode=2;
37 }
38 }
39 }catch(e){console.log('UNAVAILABLE: verify configuration, database and node');process.exitCode=3;}
40 finally{await db.end();}

```

Meaning: MATCH compares local bytes against an event from the expected package and registry on the expected network. Purpose: detecting altered snapshot bytes and incorrect proof references. Importance: Critical. It does not prove that a person actually visited, that an MRN was medically correct or that a card is unclonable.

If the worker lost its response and has no recorded digest yet, the tool reports uncertainty, not absence of any possible chain transaction. Resume exact-byte reconciliation first. Wrong configuration and network outages remain UNAVAILABLE, not tampering. Keep expected identifiers outside the database being tested so changing both a row and its local configuration is not silently accepted.

20 Executable unit tests and integration gates

backend/src/test/java/edu/upm/hsaas/RulesTest.java

JAVA / LINE NUMBERS ARE NOT SOURCE

```

1 package edu.upm.hsaas;
2 import org.junit.jupiter.api.Test;
3 import org.springframework.web.server.ResponseStatusException;
4 import static org.junit.jupiter.api.Assertions.*;
5 class RulesTest {
6     @Test void normalizesUid() {
7         assertEquals("04AABCC",Store.uid("04:aa:bb:cc"));
8     }
9     @Test void rejectsMalformedUid() {
10        assertThrows(ResponseStatusException.class,()->Store.uid("04ZZ"));
11        assertThrows(ResponseStatusException.class,()->Store.uid("040"));
12    }
13    @Test void hashIsStableAndSensitive() {
14        assertEquals(64,Store.sha("snapshot").length());
15        assertEquals(Store.sha("snapshot"),Store.sha("snapshot"));
16        assertNotEquals(Store.sha("snapshot"),Store.sha("Snapshot"));
17    }
18    @Test void categoryAllowlist() {
19        assertEquals(4,Store.CATEGORIES.size());
20        assertFalse(Store.CATEGORIES.contains("ADMIN"));
21    }
22 }

```

These small unit tests are not a substitute for database or browser tests. Run mvn test, npm run build, node syntax checks and sui move test separately. The following real-MySQL checks are mandatory before claiming the assembled reference works.

Check	Procedure and expected evidence
Atomicity	Temporarily deny audit insert in a disposable DB user; issue fails and no assignment/card/scan change commits
Duplicate command	Repeat one issue with the same key/body; same ID, one event/outbox; different body with same key gets conflict
Concurrent card issue	Two independent authenticated sessions, two overlapping commands; only one active assignment survives
Review precondition	Call issue directly with SUBMITTED/REJECTED; both fail
Role/CSRF	Staff calls admin route; browser write lacks token; both are denied
Scan context	Submit wrong actor/purpose/target/expired scan; every case fails
MRN	DEMO-0001/WARD-A valid, other pair invalid, DEMO-TIMEOUT unavailable; invalid final submit creates nothing
Return	Return same assignment twice; one return event; card becomes AVAILABLE

Check	Procedure and expected evidence
Worker recovery	Kill before execute, after execute response and before confirmation; replay uses stored bytes
Proof	Change one payload byte in an isolated test copy; local check and independent verifier reject it

For a true concurrent test, synchronize request start times and use separate sessions/transactions. Sequentially clicking twice is not a concurrency test. Save request bodies with synthetic data, status codes and final SQL counts. Browser tests must cover mobile registration, keyboard navigation, missing-reader/retry, duplicate UID and category mismatch.

Code review checklist: no TODO method bodies, every imported project file has a listing, no production credential in code, every API mutation reaches `command()`, all server state comes from the database, and all chain results pass through failure handling. Build/runtime gates remain your recorded results, not a claim inferred from this checklist.

21 Local setup with exact environment responsibilities

infra/compose.yaml

YAML / LINE NUMBERS ARE NOT SOURCE

```

1 services:
2   mysql:
3     image: mysql:8.4
4     environment:
5       MYSQL_DATABASE: hsaas
6       MYSQL_USER: hsaas_app
7       MYSQL_PASSWORD: ${DB_PASSWORD:?Set DB_PASSWORD}
8       MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD:?Set DB_ROOT_PASSWORD}
9     ports:
10      - "127.0.0.1:3306:3306"
11     volumes:
12      - hsaas_mysql:/var/lib/mysql
13 volumes:
14   hsaas_mysql:

```

Create an untracked `infra/.env` with your two different passwords. From the reference root run the following in PowerShell. Placeholder values are configuration, not shipped secrets. Replace them before running. The first command explicitly chooses the env file; Spring, Node and Java agent environments must be set separately.

POWERSHELL / LINE NUMBERS ARE NOT SOURCE

```

1 docker compose --env-file infra/.env -f infra/compose.yaml up -d
2 $env:DB_URL = 'jdbc:mysql://127.0.0.1:3306/hsaas?connectionTimeZone=UTC'
3 $env:DB_USER = 'hsaas_app'
4 $env:DB_PASSWORD = '<your local app password>'
5 $env:DEVICE_TOKEN = '<32+ random characters, shared only with reader>'
6 $env:BOOTSTRAP_PASSWORD = '<your unique 16+ character admin password>'
7 cd backend
8 mvn test
9 mvn spring-boot:run

```

In a second terminal, enter frontend, run `npm.cmd install` and `npm.cmd run dev`. Open the shown localhost URL. Login as admin using your own bootstrap password, create a staff account, set loan duration and use /register for a synthetic registration. Remove the bootstrap variable before subsequent launches.

In a third terminal set the reader environment and run from reader-agent:

POWERSHELL / LINE NUMBERS ARE NOT SOURCE

```

1 $env:API_URL = 'http://localhost:8080'
2 $env:DEVICE_TOKEN = '<same configured device token>'
3 $env:MOCK_UID = '04AABBCC'
4 cd reader-agent
5 mvn compile exec:java

```

This explicitly simulates a card for the first closed-loop test. For physical mode remove `MOCK_UID` from that process environment, set `READER_NAME` exactly and test with the vendor utility first. Stop and restart the agent when changing environment values. Never print the token in screenshots.

Worker environment uses `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `SUI_GRP_URL`, `SUI_PRIVATE_KEY`, `EXPECTED_ADDRESS`, `EXPECTED_CHAIN_ID`, `PACKAGE_ID`, `REGISTRY_ID` and

WRITER_CAP_ID. Use a dedicated restricted worker database user in deployment. The verifier needs read access and public chain identifiers only, not a private key. Use an HTTPS Testnet node endpoint and a Testnet-only wallet.

Obtain EXPECTED_CHAIN_ID from a trusted Testnet client/reference during provisioning, record it in the deployment manifest and do not automatically overwrite it on every startup. Changing network or package requires a fresh environment or explicit migration, not editing identifiers underneath pending signed transactions.

22 Deploy, back up and recover

backend/Dockerfile

DOCKERFILE / LINE NUMBERS ARE NOT SOURCE

```

1 FROM maven:3.9-eclipse-temurin-21 AS build
2 WORKDIR /src
3 COPY pom.xml .
4 COPY src ./src
5 RUN mvn -B verify
6
7 FROM eclipse-temurin:21-jre
8 WORKDIR /app
9 COPY --from=build /src/target/app.jar /app/app.jar
10 USER 10001
11 ENTRYPOINT ["java","-jar","/app/app.jar"]

```

Railway: create isolated MySQL and Backend services, select backend as root, use its Dockerfile, configure private database connectivity and the platform PORT. Set COOKIE_SECURE=true. Keep database and worker without public domains. Deploy the worker from sui-worker with npm ci and npm start and a single replica. Configure health at /actuator/health and test restart behavior.

Vercel: select frontend as root, Vite preset, npm run build and dist output. Add the following complete deployment file only after replacing the HTTPS domain. This is frontend/vercel.json; it was omitted from the local-only tree because it requires your actual deployment domain.

JSON / LINE NUMBERS ARE NOT SOURCE

```

1 {
2   "rewrites": [
3     {"source": "/api/:path*", "destination": "https://YOUR-BACKEND-DOMAIN/api/:path*"},
4     {"source": "/(.*)", "destination": "/index.html"}
5   ]
6 }

```

Verify Set-Cookie, POST/CSRF, no-store headers, deep-link reload and no localhost redirects through the real proxy. Print QR only from the deployed frontend origin. Keep the reader on the counter PC, not in the cloud. Platform plans, resource limits and costs must be checked before provisioning.

Before a release, back up MySQL and test restore to a separate database. This Bash command prompts for the password rather than exposing it as a command argument:

BASH / LINE NUMBERS ARE NOT SOURCE

```

1 mysqldump --host=DB_HOST --user=BACKUP_USER -p \
2 --single-transaction --no-tablespaces --result-file=hsaas.sql.backup hsaas

```

Do not run destructive restore commands against the only original database. Pause the worker, restore an isolated copy, check schema/history/counts/active assignments, then reconcile outbox rows with the original chain. Preserve signed bytes and signatures; do not rebuild unknown transactions casually. A lost-response row may already be on-chain.

Retry policy: inspect FAILED and attempts>=12 rows before any reset. A transport outage can resume the same saved transaction after fixing connectivity. A definitive Move failure requires a diagnosed cause, approval and a recorded new attempt; never change the event hash. Do not mark CONFIRMED manually. A fresh package/registry is a new deployment, not a way to hide conflicting commitments.

23 Annotation index and release boundary

Marker	Meaning and purpose	Importance and failure if removed
S01	Environment-supplied initial administrator secret	Critical: shared default credentials allow takeover
S02	Separate bearer-only device security chain	Critical: a reader credential could become human authority
S03	Reload active role for existing sessions	High: revoked users retain privileged sessions
D01	Generated-column active uniqueness	Critical: duplicate active card/registration allocations
D02-D03	Atomic command result, state and audit writes	Critical: retries duplicate operations or partial commits diverge
D04	Versioned exact event bytes and hash	High: later verification cannot reproduce the original commitment
B01	Final server-side MRN validation	Critical: browser checkmarks can be forged or stale
B02	Authoritative state/category checks	Critical: wrong/unavailable cards can be assigned
U01	Stable key for unknown transport outcome	Critical: network retries repeat business mutations
U02	Ignore late scan responses	High: the wrong card/context is displayed as current
R01	Single-use actor/purpose/target-bound scan	Critical: stale/cross-operation evidence is accepted
R02	Card removal before another scan	High: one held card triggers multiple jobs
M01	Writer capability bound to registry	Critical: authority is usable against the wrong registry
M02	Same-event conflict rejection	Critical: earlier commitments can be replaced
W01	Expected chain identity check	Critical: proofs are sent to the wrong network
W02	Persist signed intent before execute	Critical: lost responses create uncontrolled new intents
W03	Success plus confirmation before Confirmed	Critical: a Move failure is reported as proof success

The release boundary is deliberately explicit. This is a complete compact teaching-reference listing for the stated implementation, not a production-certified hospital system or every possible extension from the broader

planning documents. Chapter 24 adds the later requested notification and escalation behavior to this reference. Lost-card replacement, NDEF provisioning, multiple counters, detailed historic exports, persistent distributed sessions, a real Hospital API and physical door integration require separately specified workflows and tests. Existing limitations are not hidden behind stubbed methods.

Before using real data: obtain hospital-approved forms and access policy, implement data retention and purge, separate migration/runtime database accounts, restrict network access, add rate limiting/body-size limits, complete audit/secret handling review, test browser/USB/Testnet integration, and record recovery evidence. Do not treat successful PDF generation as successful application integration.

Official technical references

Spring Boot version/support baseline:

<https://docs.spring.io/spring-boot/system-requirements.html>

Spring Security CSRF behavior:

<https://docs.spring.io/spring-security/reference/servlet/exploits/csrf.html>

MySQL generated columns and constraints:

<https://dev.mysql.com/doc/refman/8.4/en/create-table-generated-columns.html>

Java PC/SC transport:

<https://docs.oracle.com/en/java/javase/21/docs/api/java.smartcardio/javax/smartcardio/package-summary.html>

Sui table API and package onboarding:

https://docs.sui.io/references/framework/sui_sui/table

<https://docs.sui.io/getting-started/onboarding/hello-world>

Mysten client, manual signing, results and event type definitions:

<https://sdk.mystenlabs.com/sui/clients/grpc>

<https://sdk.mystenlabs.com/sui/clients/executing>

<https://sdk.mystenlabs.com/sui/transactions/signing-and-execution>

<https://github.com/MystenLabs/ts-sdks/blob/main/packages/sui/src/client/types.ts>

The companion English guideline contains the full source index and deployment/hardware discussion. Vendor facts were checked against these official references; application code and design tradeoffs are original reference choices for this project.

24 Approval updates, return reminders and overdue alerts

This chapter incorporates the later requested visitor notification flow. Apply the exact edits below after chapters 01-23. The message transport in this edition is explicitly MOCK: it records intended messages without contacting WhatsApp or logging the mobile number. That permits end-to-end business and timing tests while the hospital has not approved a real WhatsApp account, templates or data handling. The operational semantics are complete in mock mode; actual provider submission and delivery callbacks are a separate integration gate.

The visitor may receive two different updates. VERIFIED can trigger APPROVED, which has no pass number because no card has been issued. After issue, PASS_ISSUED includes the public assignment reference and saved due_at. Thirty minutes before the due time, a still-open assignment can trigger RETURN_REMINDER once. Staff see overdue assignments even when the visitor declined messaging.

backend/src/main/resources/db/migration/V2__notifications.sql

SQL / LINE NUMBERS ARE NOT SOURCE

```

1 CREATE TABLE notification_jobs (
2   id BIGINT AUTO_INCREMENT PRIMARY KEY,
3   resource_id CHAR(36) NOT NULL,
4   kind VARCHAR(25) NOT NULL,
5   state VARCHAR(20) NOT NULL DEFAULT 'PENDING',
6   created_at TIMESTAMP(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
7   attempted_at TIMESTAMP(6) NULL,
8   error_code VARCHAR(60),
9   mock_text TEXT,
10  UNIQUE KEY uq_message_intent (resource_id,kind),
11  INDEX ix_message_ready (state,id),
12  CHECK (kind IN ('APPROVED','PASS_ISSUED','RETURN_REMINDER')),
13  CHECK (state IN ('PENDING','MOCK_SENT','UNKNOWN','SKIPPED'))
14 );
15 CREATE TABLE staff_followups (
16   id BIGINT AUTO_INCREMENT PRIMARY KEY,
17   assignment_id CHAR(36) NOT NULL,
18   actor VARCHAR(50) NOT NULL,
19   action VARCHAR(20) NOT NULL,
20   note VARCHAR(300) NOT NULL,
21   created_at TIMESTAMP(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
22   FOREIGN KEY (assignment_id) REFERENCES assignments(id)
23 );
```

Change Store.register in Store.java

Immediately after `clean.put("purpose",value(b,"purpose",200));` insert the following complete block. Consent to mandatory information collection and optional WhatsApp updates are separate choices. The phone must be international E.164 style when optional WhatsApp is selected.

JAVA / LINE NUMBERS ARE NOT SOURCE

```

1   boolean wa="yes".equals(b.get("whatsappOptIn"));
2   if (wa) require(clean.get("phone").matches("\\+[1-9][0-9]{7,14}"),
3     "WHATSAPP_PHONE_FORMAT");
4   clean.put("whatsappOptIn",Boolean.toString(wa));
```

Change Registration in App.tsx

Inside the registration form, insert this checkbox before the existing required notice checkbox. It starts unchecked and has its own form field; a visitor can register without it.

TSX / LINE NUMBERS ARE NOT SOURCE

```
1 <label className="check"><input type="checkbox"
2   checked={form.whatsappOptIn==='yes'}
3   onChange={e=>set({...form,whatsappOptIn:e.target.checked?'yes':'no'})}/>
4   Send me WhatsApp approval, pass and return reminders.
5   I can ask the counter to stop these messages.</label>
```

Enable scheduling in App.java

Add this import and annotation to the existing App class:

JAVA / LINE NUMBERS ARE NOT SOURCE

```
1 import org.springframework.scheduling.annotation.EnableScheduling;
2 // Add immediately before @SpringBootApplication:
3 @EnableScheduling
```

backend/src/main/java/edu/upm/hsaas/Notifications.java

This complete service runs on one Backend instance in the reference deployment. It creates unique intent rows, selects one pending message, rechecks assignment state and records a mock send. Because it never hands a request to an external provider, MOCK_SENT is a local test outcome and must never be displayed as WhatsApp delivery.

JAVA / LINE NUMBERS ARE NOT SOURCE

```
1 package edu.upm.hsaas;
2
3 import java.sql.Timestamp;
4 import java.time.*;
5 import java.time.format.DateTimeFormatter;
6 import java.util.*;
7 import org.springframework.jdbc.core.JdbcTemplate;
8 import org.springframework.scheduling.annotation.Scheduled;
9 import org.springframework.stereotype.Service;
10 import org.springframework.transaction.support.TransactionTemplate;
11
12 @Service
13 public class Notifications {
14   final JdbcTemplate db; final TransactionTemplate tx;
15   final ZoneId zone=ZoneId.of("Asia/Kuala_Lumpur");
16   public Notifications(JdbcTemplate db,TransactionTemplate tx) {
17     this.db=db;this.tx=tx;
18   }
19   @Scheduled(fixedDelay=60000,initialDelay=5000)
20   public void tick() {
21     tx.executeWithoutResult(status -> {
22       db.queryForObject("SELECT id FROM gate WHERE id=1 FOR UPDATE",Integer.class);
23       // [N01] Approval contains no card or due time and is skipped after issue.
24       db.update("""
25         INSERT IGNORE INTO notification_jobs(resource_id,kind)
26         SELECT r.id,'APPROVED' FROM registrations r
27         WHERE r.state='VERIFIED'
28         AND JSON_UNQUOTE(JSON_EXTRACT(r.data,'$.whatsappOptIn'))='true'
29         AND NOT EXISTS (SELECT 1 FROM assignments a
```


JAVA / LINE NUMBERS ARE NOT SOURCE

```
30     WHERE a.registration_id=r.id)
```

JAVA / LINE NUMBERS ARE NOT SOURCE

```
31     """);
32     db.update("""
33     INSERT IGNORE INTO notification_jobs(resource_id,kind)
34     SELECT a.id,'PASS_ISSUED' FROM assignments a
35     JOIN registrations r ON r.id=a.registration_id
36     WHERE JSON_UNQUOTE(JSON_EXTRACT(r.data,'$.whatsappOptIn'))='true'
37     """);
38     // [N02] UTC database clock; no reminder after recorded return.
39     db.update("""
40     INSERT IGNORE INTO notification_jobs(resource_id,kind)
41     SELECT a.id,'RETURN_REMINDER' FROM assignments a
42     JOIN registrations r ON r.id=a.registration_id
43     WHERE a.returned_at IS NULL AND a.due_at>CURRENT_TIMESTAMP
44     AND a.due_at<=DATE_ADD(CURRENT_TIMESTAMP,INTERVAL 30 MINUTE)
45     AND JSON_UNQUOTE(JSON_EXTRACT(r.data,'$.whatsappOptIn'))='true'
46     """);
47     var waiting=db.queryForList("""
48     SELECT id,resource_id,kind FROM notification_jobs
49     WHERE state='PENDING' ORDER BY id LIMIT 1
50     """);
51     if(waiting.isEmpty())return;
52     var j=waiting.getFirst();
53     String kind=(String)j.get("kind"),resource=(String)j.get("resource_id");
54     if(kind.equals("APPROVED")) {
55         var rows=db.queryForList("SELECT state FROM registrations WHERE id=?",resource);
56         var issued=db.queryForList("SELECT id FROM assignments WHERE registration_id=?",resource);
57         if(rows.isEmpty() || !"VERIFIED".equals(rows.getFirst().get("state")))
58             || !issued.isEmpty()) {
59             skip(j);return;
60         }
61     }
```

JAVA / LINE NUMBERS ARE NOT SOURCE

```
61     acceptMock(j,"Your visitor request "+resource+" is approved. "
62     +"Please collect the pass at the counter. No pass is issued yet.");
63     return;
64 }
65 var rows=db.queryForList("""
66     SELECT a.id,a.due_at,a.returned_at FROM assignments a WHERE a.id=?
67     """,resource);
68 if(rows.isEmpty()){skip(j);return;}
69 var assignment=rows.getFirst();
70 if(kind.equals("PASS_ISSUED") && assignment.get("returned_at")!=null) {
71     skip(j);return;
72 }
73 if(kind.equals("RETURN_REMINDER") && assignment.get("returned_at")!=null) {
74     skip(j);return;
75 }
76 Instant due=((Timestamp)assignment.get("due_at")).toInstant();
77 String local=DateTimeFormatter.ofPattern("dd MMM yyyy, h:mm a '(MYT, UTC+8)'")
78     .withZone(zone).format(due);
79 String pass="HSAAS-"+resource;
80 String body=kind.equals("PASS_ISSUED")
81     ? "Pass "+pass+" was issued. Return it to the counter by "+local+"."
82     : "Reminder: return pass "+pass+" to the counter by "+local+".";
83 acceptMock(j,body);
84 }));
```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

85 }
86 void skip(Map<String,Object> j) {
87     db.update("UPDATE notification_jobs SET state='SKIPPED' WHERE id=?",j.get("id"));
88 }
89 void acceptMock(Map<String,Object> j,String body) {
90     // [N03] No mobile number or message body in application logs.

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

91     db.update("
92     UPDATE notification_jobs SET state='MOCK_SENT',attempted_at=CURRENT_TIMESTAMP,
93     mock_text=? WHERE id=?
94     """,body,j.get("id"));
95 }
96 public Map<String,Object> overdue() {
97     var rows=db.queryForList("
98     SELECT a.id AS pass_id,a.uid,a.due_at,
99     TIMESTAMPDIFF(MINUTE,a.due_at,CURRENT_TIMESTAMP) AS minutes_late
100     FROM assignments a WHERE a.returned_at IS NULL
101     AND a.due_at<CURRENT_TIMESTAMP ORDER BY a.due_at LIMIT 100
102     """);
103     // [N04] Staff alert depends on open assignments, not message delivery.
104     return Map.of("count",rows.size(),"items",rows);
105 }
106 public Map<String,Object> followup(String id,String actor,Map<String,String> b) {
107     return tx.execute(status -> {
108         db.queryForObject("SELECT id FROM gate WHERE id=1 FOR UPDATE",Integer.class);
109         var a=db.queryForList("
110         SELECT id FROM assignments WHERE id=? AND returned_at IS NULL
111         AND due_at<CURRENT_TIMESTAMP
112         """,id);
113         Store.require(!a.isEmpty(),"NOT_OVERDUE");
114         String action=Store.value(b,"action",20),note=Store.value(b,"note",300);
115         Store.require(Set.of("CONTACTED","ESCALATED","EXTENDED_PENDING")
116             .contains(action),"BAD_FOLLOWUP_ACTION");
117         db.update("
118         INSERT INTO staff_followups(assignment_id,actor,action,note)
119         VALUES (?,?,,?)
120         """,id,actor,action,note);

```

JAVA / LINE NUMBERS ARE NOT SOURCE

```

121     return Map.of("ok",true);
122 });
123 }
124 }

```

[N01] High: an approval notification can be queued only while the request is verified. The issue template is separate because pass reference and deadline exist only after issue.

[N02] Critical: one reminder intent per assignment and no reminder for a recorded return. There remains a short unavoidable race between a real provider submission and an in-person return; a production provider adapter should minimize that interval and avoid claiming a hard guarantee that no late message can ever arrive.

[N03] High: the current adapter is MOCK. For a real provider, create a separate `WhatsAppSender` with approved business account credentials and reviewed utility templates, store provider message ID, consume status callbacks and distinguish accepted, delivered, failed and unknown. Never equate an HTTP acceptance with delivery. Do not send raw card UID, MRN or ward in templates. WhatsApp's [messaging policy](<https://whatsappbusiness.com/policy/>) requires permission to contact the recipient and approved

templates for business-initiated messages.

[N04] Critical: the counter staff alert works even if the visitor opted out, lacks WhatsApp or delivery failed. Staff follow-up records an action; it does not itself close the overdue assignment. A physical return closes it through the existing return command.

Add staff endpoints to Api.java

Add `final Notifications notifications;` and change the constructor to `public Api(Store s, Notifications notifications) { this.s=s;this.notifications=notifications; }`. Add the following methods within the existing class:

JAVA / LINE NUMBERS ARE NOT SOURCE

```
1 @GetMapping("/api/staff/overdue") Object overdue() {
2     return notifications.overdue();
3 }
4 @PostMapping("/api/staff/overdue/{id}/followup") Object followup(
5     @PathVariable String id,@RequestBody Map<String,String> b,
6     @RequestHeader("Idempotency-Key") String key,Authentication a) {
7     return run(a,"followup:"+id,key,b,
8         () -> notifications.followup(id,a.getName(),b));
9 }
```

Add staff alert to App.tsx

Add `const [overdue,setOverdue]=useState<Row>({count:0,items:[]});` with the other state declarations. Inside the `load()` function, before loading the selected tab, add `const late=await api('/staff/overdue');if(live)setOverdue(late);`. Immediately after the workspace `<nav>` add this panel:

TSX / LINE NUMBERS ARE NOT SOURCE

```
1 {overdue.count>0&&<section className="notice" role="alert">
2     <h2>{overdue.count} overdue pass(es)</h2>
3     <p>Check the due time and contact the visitor through approved channels.</p>
4     <Table rows={overdue.items} action={r=><Button onClick={()=>{
5         const note=prompt('Record follow-up note');
6         if(note)void action()==>api(`/staff/overdue/${r.pass_id}/followup`,
7             {action:'CONTACTED',note});
8     }}>Record follow-up</Button></>
9 </section>
```

For a multi-page staff dashboard, fetch `/api/staff/overdue` with its own five-second query even when Settings or Users is open. The compact UI refreshes the alert when tabs/actions refresh and while operational tabs poll. The alert count is capped to the first 100 rows in this reference; add a count query and pagination for larger deployments.

Acceptance tests: VERIFIED triggers an approval intent without pass/deadline; successful issue triggers PASS_ISSUED with the assignment reference and precise MYT date/time; a pending return triggers one reminder in the 30-minute window; returning before the window prevents that reminder; overdue rows alert staff regardless of opt-in; follow-up logs actor/action/note; no raw UID or patient fields appear in outbound text. The mock sender marks MOCK_SENT only. Obtain hospital approval, an official WhatsApp Business Platform account, approved templates and delivery callbacks before enabling actual messages.